Contents

Object Oriented Programming Languages

Complete Topic List

1. Classes and Objects

- Classes and objects, instantiation
- Comparing and copying of objects
- Testing

2. Encapsulation and Members

- Encapsulation, members, and constructors
- Instance and static members

- Information hiding and visibility modifiers

3. Polymorphism and Type Systems

- Overloading
- Inheritance and multiple inheritance
- Subtyping, static and dynamic types, and type checking
- Overriding and dynamic binding
- Interfaces
- Generics
- Subtype and parametric polymorphism

4. Memory Management

- Memory management and garbage collection

5. Program Structure

- Program structure, packages, and compilation units

Chapter Files

- `chapters/chap1_classes_objects.md`
- `chapters/chap2_encapsulation_members.md`
- `chapters/chap3_polymorphism_types.md`
- `chapters/chap4_memory_management.md`
- `chapters/chap5_program_structure.md` # Chapter 1: Classes and Objects

1.1 Classes and Objects, Instantiation

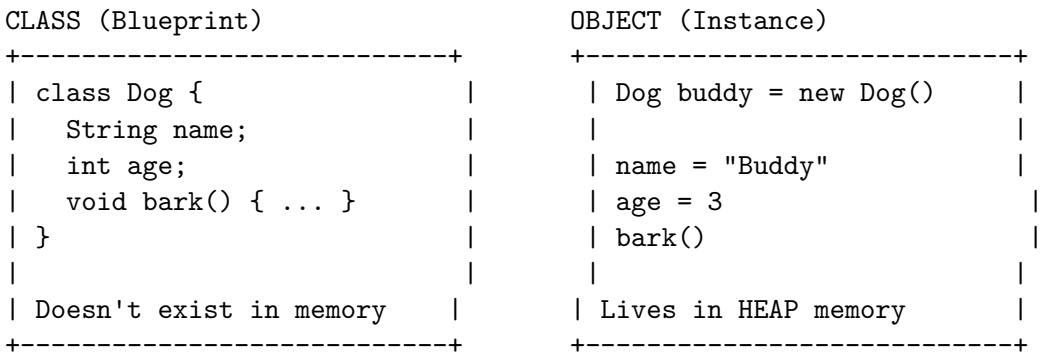
Definition / Theoretical Background

Class: A blueprint or template that defines the structure and behavior of objects. It specifies: - **Fields** (attributes/properties): Data that objects store - **Methods** (behaviors): Operations that objects can perform

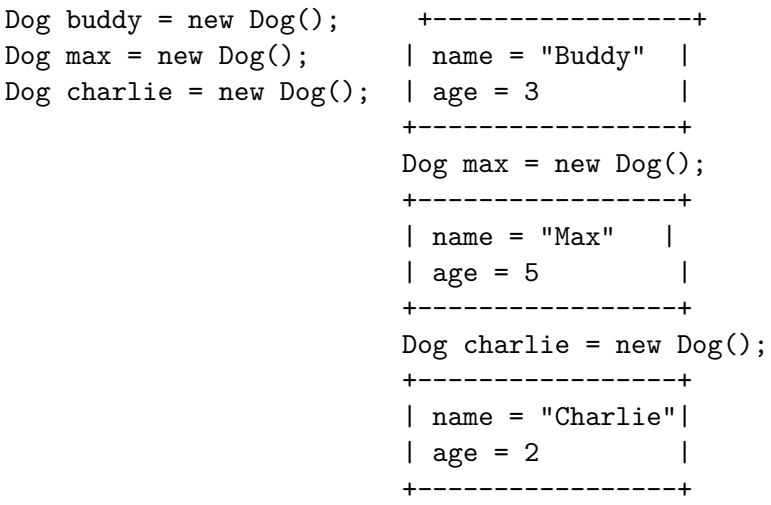
Object: An instance of a class - a concrete entity created from the blueprint.

Concept	What It Is	Example
Class	Blueprint	`class Dog { ... }`
Object	Instance of class	`Dog buddy = new Dog();`
Instantiation	Creating object from class	`new Dog()`
State	Values in object's fields	`buddy.name = "Buddy"`
Identity	Unique identifier	Each object has unique address
Behavior	Methods object can perform	`buddy.bark()`

Class vs Object Visualization



You can create MANY objects from ONE class!



Declaration and Instantiation

```java
// Class definition
class Dog {
    // Instance variables (fields)
    String name;
    int age;

    // Constructor
    Dog(String name, int age) {
        this.name = name;
        this.age = age;
    }

    // Methods
    void bark() {
        System.out.println(name + " says Woof!");
```

```
    }
}

// Instantiation and use
public class Main {
    public static void main(String[] args) {
        Dog buddy = new Dog("Buddy", 3);  // Create object
        Dog max = new Dog("Max", 5);

        buddy.bark();  // "Buddy says Woof!"
        max.bark();    // "Max says Woof!"

        // Objects have independent state
        buddy.age = 4;  // Only buddy's age changes
        System.out.println(max.age);  // Still 5!
    }
}
```

Java

```
# Class definition (Python)
class Dog:
    # Constructor (special method)
    def __init__(self, name, age):
        self.name = name  # self refers to the object
        self.age = age

    def bark(self):
        print(f"{self.name} says Woof!")

# Instantiation
buddy = Dog("Buddy", 3)
max = Dog("Max", 5)

buddy.bark()  # "Buddy says Woof!"
max.bark()    # "Max says Woof!"
```

Python

```
class Dog {
public:
    string name;   // Instance variables
    int age;

    // Constructor
    Dog(string name, int age) : name(name), age(age) {}
```

```
    void bark() {
        cout << name << " says Woof!" << endl;
    }
};

int main() {
    Dog* buddy = new Dog("Buddy", 3);  // Heap allocation
    Dog max("Max", 5);                  // Stack allocation

    buddy->bark();  // "Buddy says Woof!"
    max.bark();     // "Max says Woof!"

    delete buddy;   // Don't forget to free heap memory!
}
```

C++

The `new` Keyword (Instantiation)

```
Dog buddy = new Dog(...);
```

What `new` does: 1. Allocates memory for the object (usually on the heap) 2. Initializes fields to default values (0, null, false) 3. Calls the constructor method 4. Returns a reference to the object

```
new Dog("Buddy", 3)
      |
      v
+---------------------------------+
| 1. Allocate memory (heap)       |
| 2. Initialize fields:            |
|    name = null                   |
|    age = 0                       |
| 3. Call constructor:             |
|    name = "Buddy"                |
|    age = 3                       |
| 4. Return reference (address)    |
+---------------------------------+
```

Default Values

```
class Example {
    int num;           // Default: 0
    double decimal;    // Default: 0.0
    boolean flag;      // Default: false
    String text;       // Default: null
    int[] arr;         // Default: null
    Object obj;        // Default: null
```

```
    // Constructor to initialize
    Example() {
        num = 10;  // Now it's 10
        // text still null if not initialized
    }
}
```

Default values table:

Type	Default Value
byte, short, int, long	0
float, double	0.0
char	‘ŏ000’ (null character)
boolean	false
Object reference	null

Memory Where Objects Live

```
STACK (local variables)          HEAP (objects)
+---------------------+         +---------------------+
| Dog buddy ----------+-------->| Dog object          |
| (reference)         |         | name: "Buddy"       |
+---------------------+         | age: 3              |
                                 +---------------------+
```

In Java, Python, C#: - **Objects live on the Heap** (managed memory) - **References live on the Stack** (variables point to heap)

In C++: - Objects can live on **Stack** (if declared locally) or **Heap** (if `new`)

Edge Cases & Traps

Trap 1: Reference vs Object Confusion

```
Dog d1 = new Dog("Buddy", 3);
Dog d2 = d1;  // Copies REFERENCE, not object!

// Now d1 and d2 point to SAME object!
d2.name = "Max";
System.out.println(d1.name);  // "Max"! Changed because same object!

// For independent copies, need to copy explicitly:
Dog d3 = new Dog(d1.name, d1.age);  // Creates NEW object
```

Trap 2: Null Reference

```
Dog d;  // d is null (no object!)
d.bark();  // NullPointerException! Object doesn't exist!
```

```
// Must initialize first:
d = new Dog("Buddy", 3);
d.bark();  // Now works
```

Trap 3: Uninitialized Fields

```
class Example {
    int x;  // Default 0, not uninitialized garbage!
}
// x is 0, not random garbage
// Unlike C/C++ local variables!
```

Trap 4: Class is a Type, Object is an Instance

```
Dog d = new Dog();  // Dog is type/class, d is instance
// You cannot do: new Dog.bark()  (wrong!)
// Must: d.bark() (using object instance)
```

Examples

```
class BankAccount {
    // Fields
    String accountNumber;
    String ownerName;
    double balance;

    // Constructor
    BankAccount(String accNum, String name, double initialBalance) {
        accountNumber = accNum;
        ownerName = name;
        balance = initialBalance;
    }

    // Methods
    void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
        }
    }

    void withdraw(double amount) {
        if (amount <= balance) {
            balance -= amount;
        } else {
            System.out.println("Insufficient funds!");
        }
    }
```

```
    void display() {
        System.out.println(ownerName + ": $" + balance);
    }
}

public class Main {
    public static void main(String[] args) {
        BankAccount acc1 = new BankAccount("12345", "Alice", 1000);
        BankAccount acc2 = new BankAccount("67890", "Bob", 500);

        acc1.deposit(500);   // Alice: $1500
        acc2.withdraw(100);  // Bob: $400
        acc1.display();       // Alice: $1500.0
        acc2.display();       // Bob: $400.0

        // Each object has its OWN state
        System.out.println(acc1.balance);  // 1500.0
        System.out.println(acc2.balance);  // 400.0
    }
}
```

Example 1: Bank Account Class

```
class Rectangle:
    def __init__(self, width, height):
        self.width = width
        self.height = height

    def area(self):
        return self.width * self.height

    def perimeter(self):
        return 2 * (self.width + self.height)

    def scale(self, factor):
        self.width *= factor
        self.height *= factor

# Create two rectangle objects
r1 = Rectangle(5, 3)
r2 = Rectangle(4, 2)

print(f"r1 area: {r1.area()}")  # 15
print(f"r2 area: {r2.area()}")  # 8

r1.scale(2)  # Scale r1 by 2
```

```
print(f"r1 area after scale: {r1.area()}")  # 60
print(f"r2 unchanged: {r2.area()}")  # Still 8!
```

Example 2: Rectangle Class

```
class Student {
    String name;
    int age;
    String major;

    // Constructor 1: Full
    Student(String name, int age, String major) {
        this.name = name;
        this.age = age;
        this.major = major;
    }

    // Constructor 2: Default major
    Student(String name, int age) {
        this.name = name;
        this.age = age;
        this.major = "Undeclared";
    }

    // Constructor 3: Default values
    Student() {
        this.name = "Unknown";
        this.age = 0;
        this.major = "Undeclared";
    }
}

// Usage
Student s1 = new Student("Alice", 20, "CS");          // Full
Student s2 = new Student("Bob", 19);                   // Default major
Student s3 = new Student();                            // All defaults
```

Example 3: Constructor Overloading (Multiple Constructors)

The `this` Reference

The `this` keyword is a reference to the **current object** - the object on which the method is being called.

```
class Person {
    private String name;
    private int age;
```

```
    // this helps distinguish field from parameter
    Person(String name, int age) {
        this.name = name;   // this.name = field, name = parameter
        this.age = age;     // this.age = field, age = parameter
    }

    // this can be used to return the object itself (for chaining)
    Person setName(String name) {
        this.name = name;
        return this;  // Returns the same object
    }

    Person setAge(int age) {
        this.age = age;
        return this;
    }

    // Chain method calls
    public static void main(String[] args) {
        Person p = new Person("Alice", 20);
        p.setName("Bob").setAge(25);  // Method chaining!
    }
}
```

Common uses of `this`:

Use	Example	When Needed
Disambiguate field/parameter	`this.x = x`	Parameter has same name as field
Return current object	`return this`	For method chaining
Pass current object	`callback(this)`	When object needs to reference itself
Constructor chaining	`this(args)`	Call another constructor in same class

this() for Constructor Chaining:

```
class Student {
    String name;
    int age;
    String major;

    // Chain to the full constructor
    Student(String name, int age) {
        this(name, age, "Undeclared");  // Calls Student(String, int, String)
    }

    Student(String name, int age, String major) {
        this.name = name;
        this.age = age;
```

```
        this.major = major;
    }
}
```

Important Note: `this()` must be the **first statement** in constructor!

Immutability

An **immutable object** is an object whose state **cannot be changed** after creation.

```
// Immutable class example
final class Person {
    private final String name;  // final = can't reassign
    private final int age;

    // No setters! Only getters and constructor
    Person(String name, int age) {
        this.name = name;   // Can't change after construction
        this.age = age;
    }

    // Only getters, no setters
    public String getName() { return name; }
    public int getAge() { return age; }
}

// Usage
Person p = new Person("Alice", 25);
String n = p.getName();  // Can read, can't modify
// p.setAge(26);         // ERROR! No setter method exists!
```

Why Immutability Matters:

```
Benefits of Immutable Objects:
1. Thread-safe - No synchronization needed!
2. Cacheable - Can safely reuse instances
3. No defensive copies needed
4. Building blocks for reliable code

Examples of immutable types:
- String in Java
- int, double primitives
- LocalDate in Java 8
- tuple in Python
```

Creating Immutable Classes:

```
public final class ImmutablePoint {
    private final int x;
    private final int y;
```

```
    // No setters! Only constructor and getters
    public ImmutablePoint(int x, int y) {
        this.x = x;
        this.y = y;
    }

    public int getX() { return x; }
    public int getY() { return y; }

    // Returns NEW point, doesn't modify this
    public ImmutablePoint translate(int dx, int dy) {
        return new ImmutablePoint(x + dx, y + dy);
    }
}

// Usage
ImmutablePoint p1 = new ImmutablePoint(1, 2);
ImmutablePoint p2 = p1.translate(3, 4);  // Creates NEW point, p1 unchanged
```

The Problem with Mutable Objects:

```
// Mutable version - problematic!
class MutablePoint {
    int x, y;
    MutablePoint(int x, int y) { this.x = x; this.y = y; }
    void setX(int x) { this.x = x; }
    void setY(int y) { this.y = y; }
}

// Problem: If stored in collection, can be modified from outside!
MutablePoint p = new MutablePoint(1, 2);
List<MutablePoint> points = new ArrayList<>();
points.add(p);
points.get(0).setX(999);  // p.x is now 999! Original modified!

// Immutable version - safe!
ImmutablePoint p1 = new ImmutablePoint(1, 2);
points.add(p1);
// No way to modify p1 after creation!
```

1.2 Comparing and Copying Objects

Comparing Objects

Reference Equality vs Value Equality

Type	What It Checks	Syntax
Reference equality	Same memory location?	`==` in Java for references
Value equality	Same content?	`.equals()` in Java

```
String s1 = new String("hello");
String s2 = new String("hello");

// Reference comparison (are they the same object?)
System.out.println(s1 == s2);           // false (different objects)

// Value comparison (do they have same content?)
System.out.println(s1.equals(s2));      // true (same content!)

// BUT: String literals are pooled!
String s3 = "hello";
String s4 = "hello";

System.out.println(s3 == s4);           // true (same pooled object!)
System.out.println(s3.equals(s4));     // true (same content too)
```

```
class Point {
    int x, y;
    Point(int x, int y) { this.x = x; this.y = y; }
}

Point p1 = new Point(1, 2);
Point p2 = new Point(1, 2);

System.out.println(p1 == p2);       // false! Different objects!
System.out.println(p1.equals(p2)); // false! equals() inherited from Object
                                    // does reference comparison by default!

// You need to OVERRIDE equals()!
class Point {
    int x, y;
    Point(int x, int y) { this.x = x; this.y = y; }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;  // Same reference
        if (!(obj instanceof Point)) return false;
        Point other = (Point) obj;
        return this.x == other.x && this.y == other.y;
    }
}
```

Why == Doesn't Work for Value Comparison

```python
# Python: == does VALUE comparison by default!
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

p1 = Point(1, 2)
p2 = Point(1, 2)
p3 = p1

print(p1 == p2)  # True! (Python's == compares values by default)
print(p1 == p3)  # True! (Same object, same value)

# is checks identity (like reference equality in Java)
print(p1 is p2)  # False! Different objects
print(p1 is p3)  # True! Same object
```

Python Comparison

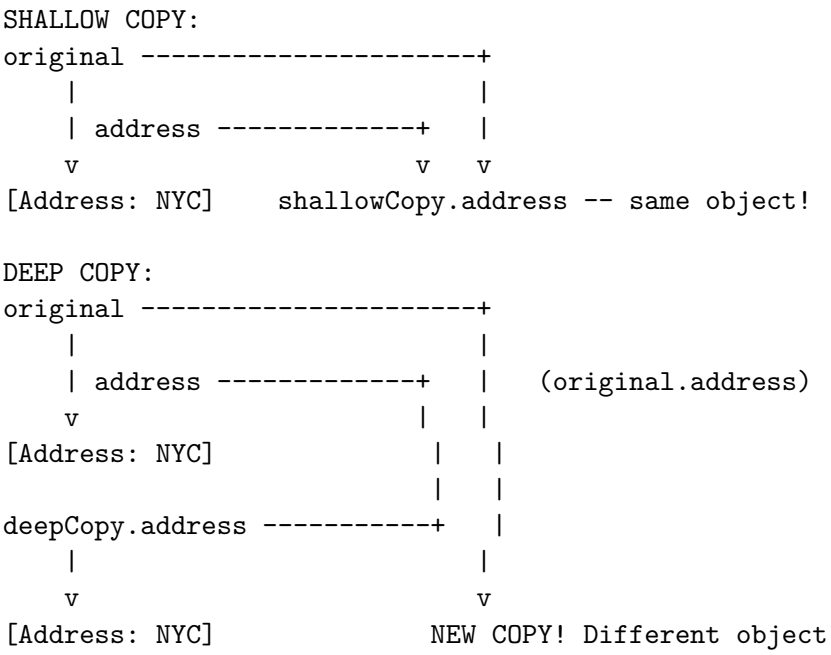
Copying Objects

```java
class Address {
    String city;
    Address(String city) { this.city = city; }
}

class Person {
    String name;
    Address address;  // Reference type field!

    Person(String name, Address address) {
        this.name = name;
        this.address = address;
    }
}

// Original
Address addr = new Address("NYC");
Person original = new Person("Alice", addr);

// Shallow copy - copies reference, not object!
Person shallowCopy = original;
// shallowCopy.address points to SAME Address object as original!
```

```
// Deep copy - copies the referenced objects too!
Person deepCopy = new Person(original.name, new Address(original.address.city));
// deepCopy.address is a NEW object with same values!
```

Shallow Copy vs Deep Copy

Shallow vs Deep Copy Visualization




```
class Person {
    String name;
    int age;

    // Regular constructor
    Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    // Copy constructor
    Person(Person other) {
        this.name = other.name;
        this.age = other.age;
    }
}

// Using copy constructor
Person original = new Person("Alice", 25);
Person copy = new Person(original);  // Creates NEW object with same values!
```

Copy Constructors

```
import copy

class Person:
    def __init__(self, name, address):
        self.name = name
        self.address = address  # Could be object reference

# Original
p1 = Person("Alice", "NYC")

# Shallow copy - nested objects shared
p2 = copy.copy(p1)

# Deep copy - nested objects also copied
p3 = copy.deepcopy(p1)

# p1 and p2 are different objects, but their address field might be same object
# p3 is completely independent
```

Python Copy Module

Edge Cases & Traps

Trap 1: == on Objects Compares References, Not Values

```
String a = new String("test");
String b = new String("test");
if (a == b) {  // FALSE! Different objects!
    // This block never executes!
}
if (a.equals(b)) {  // TRUE! Same content!
    // This block executes!
}
```

[WARNING] **Professor Trap**: Always use `.equals()` for string value comparison!

Trap 2: Mutable Default Arguments (Python)

```
# DANGER! This is a common Python trap!
class BadList:
    def __init__(self, items=[]):  # Mutable default!
        self.items = items

lst1 = BadList()
lst1.items.append(1)
```

```python
lst2 = BadList()
print(lst2.items)  # [1]! Shared default list!

# CORRECT:
def __init__(self, items=None):
    if items is None:
        items = []  # Create new list each time
```

Trap 3: Shallow Copy with Nested Objects

```java
class Node {
    Node next;
    int value;
}

Node n1 = new Node();
n1.value = 1;
Node n2 = new Node();
n2.value = 2;
n1.next = n2;

// Shallow copy of n1
Node copy = new Node();
copy.value = n1.value;
copy.next = n1.next;  // copy.next points to SAME n2!

// Now modifying n2 affects both original and copy!
```

Trap 4: Forgetting to Override equals()

```java
class Point {
    int x, y;
    // No equals() override!
}

Point p1 = new Point(1, 2);
Point p2 = new Point(1, 2);

// These are both false!
System.out.println(p1 == p2);      // false (different objects)
System.out.println(p1.equals(p2)); // false (inherited == checks references!)
```

[WARNING] **Professor Trap**: If you override equals(), also override hashCode() for Maps/Sets!

Examples

```java
class Point {
    private int x, y;
```

```
    public Point(int x, int y) {
        this.x = x;
        this.y = y;
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;  // Same reference
        if (obj == null || getClass() != obj.getClass()) return false;
        Point other = (Point) obj;
        return x == other.x && y == other.y;
    }

    @Override
    public int hashCode() {
        return 31 * x + y;  // Consistent with equals()
    }
}

// Now:
Point p1 = new Point(1, 2);
Point p2 = new Point(1, 2);
System.out.println(p1.equals(p2));  // true! Same values!
System.out.println(p1 == p2);       // false! Different objects!
```

Example 1: Proper equals() Implementation

```
class Person implements Cloneable {
    String name;
    int age;

    Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    @Override
    protected Object clone() {
        return new Person(this.name, this.age);  // Deep copy
    }
}

Person original = new Person("Alice", 25);
Person cloned = (Person) original.clone();

System.out.println(original == cloned);       // false
```

```
System.out.println(original.name.equals(cloned.name));  // true
```

Example 2: Clone Method

1.3 Testing

Testing Concepts

Why test? - Find bugs before users do - Verify functionality works correctly - Enable confident refactoring - Document expected behavior

Unit Testing Basics

```
// Simple unit test example (JUnit)
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class Calculator {
    int add(int a, int b) { return a + b; }
    int divide(int a, int b) {
        if (b == 0) throw new ArithmeticException("Division by zero");
        return a / b;
    }
}

class CalculatorTest {
    @Test
    void testAdd() {
        Calculator calc = new Calculator();
        assertEquals(5, calc.add(2, 3));
        assertEquals(0, calc.add(-1, 1));
    }

    @Test
    void testDivide() {
        Calculator calc = new Calculator();
        assertEquals(2, calc.divide(10, 5));
    }

    @Test
    void testDivideByZero() {
        Calculator calc = new Calculator();
        assertThrows(ArithmeticException.class, () -> calc.divide(1, 0));
    }
}
```

Testing Terminology

Term	Meaning
Unit test	Tests single method/class
Integration test	Tests multiple components together
Test fixture	Setup code before tests
Assertion	Check that expected == actual
Mock	Fake object for testing dependencies
TDD	Test-Driven Development (write tests first)

White-Board Example: Testing OOP Concepts

```
Question: What is the output?

Dog d1 = new Dog("Buddy", 3);
Dog d2 = d1;           // What is d2?
d2.setAge(4);
System.out.println(d1.getAge());

Answer: 4

Explanation:
1. d1 created with age 3
2. d2 = d1 means d2 points to SAME object as d1
3. d2.setAge(4) modifies the shared object
4. d1.getAge() returns 4

This tests understanding of REFERENCE semantics!
```

Practice Questions

MCQ 1: What does `new` do in Java? - A) Declares a new variable - B) Allocates memory and calls constructor - C) Copies an existing object - D) Deletes an object

MCQ 2: Two objects created with `new Dog()` and same values - what is `==`? - A) true - B) false - C) Depends on equals() method - D) Compile error

MCQ 3: What is shallow copy? - A) Copies all fields including references - B) Copies only primitive fields - C) Creates new referenced objects - D) Used for value types only

Short Answer: Explain why `String s1 = new String("hi") == String s2 = new String("hi")` returns false.

Board Coding: Write a class `Circle` with radius field, constructor, and method `getArea()`. Then write tests for it.

Oral Explanation: What's the difference between `==` and `.equals()` for objects? When would you use each?

Solutions

MCQ 1: B) Allocates memory and calls constructor

MCQ 2: B) false - different objects in memory

MCQ 3: A) Copies all fields, but reference fields point to same objects

Short Answer: `new` creates a new object in heap memory. Even though both strings have content "hi", they are different objects at different memory locations, so `==` compares references (addresses) and returns false. Use `.equals()` for content comparison.

Cheat-Sheet

Concept	Key Points
Class	Blueprint/template defining structure and behavior
Object	Instance of class, lives in heap memory
Instantiation	`new ClassName()` allocates and initializes
Constructor	Initializes new object, has same name as class
Reference	Variable that points to object in heap
null	Reference pointing to nothing
==	Compares references (memory addresses)
equals()	Compares object values (override for custom comparison)
Shallow copy	Copies object, shared references inside
Deep copy	Copies object AND all nested objects

Review Checklist

- ☐ **Class vs Object**: Can you explain the difference?
- ☐ **Instantiation**: Do you understand what `new` does?
- ☐ **Reference semantics**: Can you trace reference assignments?
- ☐ **Comparison**: Do you know when to use == vs equals()?
- ☐ **Copying**: Can you explain shallow vs deep copy?
- ☐ **Testing**: Can you write basic unit tests? # Chapter 2: Encapsulation and Members

2.1 Encapsulation

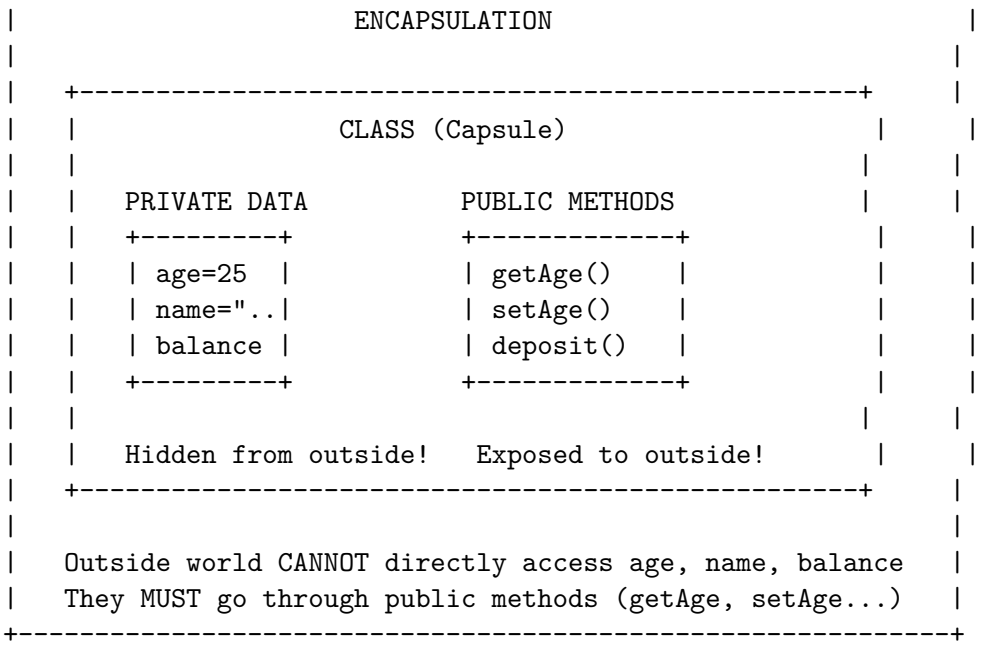
Definition / Theoretical Background

Encapsulation is the bundling of data (fields) and methods (behaviors) into a single unit (class), and restricting access to some of the object's internal components.

Key Benefits: 1. **Data hiding** - Hide internal implementation details 2. **Protection** - Prevent external code from corrupting state 3. **Modularity** - Objects are self-contained, independent units 4. **Flexibility** - Can change internal implementation without affecting users

The Encapsulation Principle

```
+------------------------------------------------------------+
```



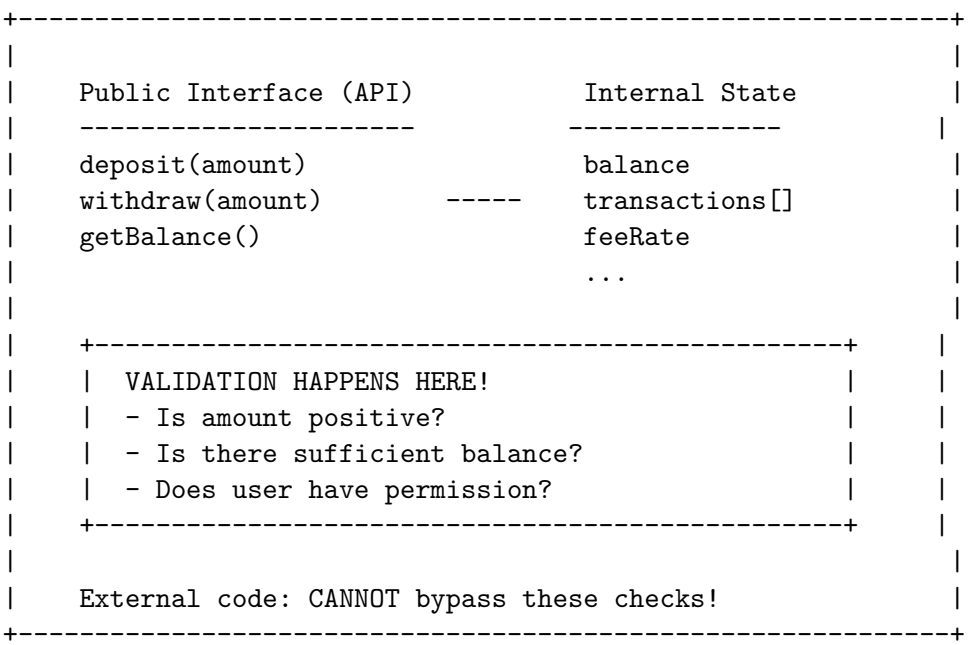
Why Encapsulation Matters

```
// WITHOUT encapsulation (BAD!)
class BankAccountBad {
    double balance;  // PUBLIC! Anyone can modify!

    BankAccountBad(double balance) {
        this.balance = balance;  // No validation!
    }
}

// Anyone can do dangerous things:
account.balance = -1000000;  // Negative balance! No validation!
account.balance = Double.MAX_VALUE;  // Overflow possible!
```

```
// WITH encapsulation (GOOD!)
class BankAccountGood {
    private double balance;  // PRIVATE!

    BankAccountGood(double initialBalance) {
        if (initialBalance < 0) {
            throw new IllegalArgumentException("Cannot be negative!");
        }
        this.balance = initialBalance;
    }

    public void deposit(double amount) {
        if (amount <= 0) {
            throw new IllegalArgumentException("Amount must be positive!");
```

```
        }
        balance += amount;
    }

    public void withdraw(double amount) {
        if (amount <= 0) {
            throw new IllegalArgumentException("Amount must be positive!");
        }
        if (amount > balance) {
            throw new IllegalArgumentException("Insufficient funds!");
        }
        balance -= amount;
    }

    public double getBalance() {
        return balance;  // Read-only access (no setter!)
    }
}

// NOW: Balance is protected! All operations validated!
// account.balance = -1000000;  // ERROR! Can't access private field!
// Must use: account.deposit() which validates!
```

The Protective Barrier



2.2 Members (Fields and Methods)

Instance Members vs Static Members

Aspect	Instance Member	Static Member
Belongs to	Each object (instance)	Class itself
Memory	Each object has its own copy	Single copy shared
Access	`object.member`	`ClassName.member`
Lifetime	As long as object exists	Whole program
Keyword	(none)	`static`

Instance Members

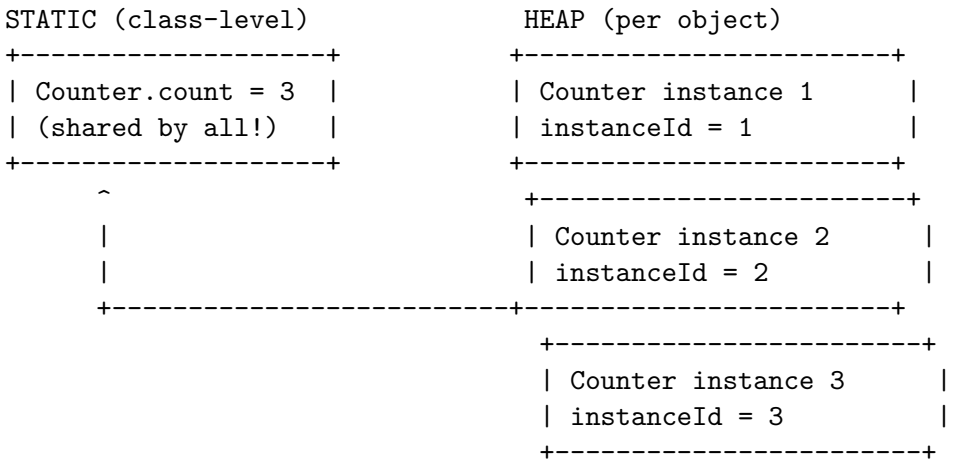
```
class Dog {
    String name;  // Instance field - each Dog has its own name
    int age;      // Instance field - each Dog has its own age

    void bark() {  // Instance method - operates on specific instance
        System.out.println(name + " says Woof!");
    }
}

Dog buddy = new Dog();
Dog max = new Dog();

buddy.name = "Buddy";  // buddy's name
max.name = "Max";       // max's name (independent!)

buddy.bark();  // "Buddy says Woof!" - operates on buddy
max.bark();    // "Max says Woof!" - operates on max
```

Static Members

```
class Counter {
    private static int count = 0;  // Static field - ONE copy for class
    private int instanceId;       // Instance field - one per object

    Counter() {
        instanceId = ++count;  // Increment shared count
    }

    public static int getCount() {  // Static method
        return count;  // Can only access static members
    }

    public int getInstanceId() {   // Instance method
        return instanceId;  // Can access both static and instance
```

```
    }
}

Counter c1 = new Counter();  // instanceId=1, count=1
Counter c2 = new Counter();  // instanceId=2, count=2
Counter c3 = new Counter();  // instanceId=3, count=3

// All objects share the SAME count!
System.out.println(Counter.getCount());  // 3 (via class name)
// c1.getCount() also works but convention is ClassName.method()
```

Static Field Memory Visualization



When to Use Static?

```
// Use STATIC when:
// 1. Value is shared by ALL instances
class MathUtils {
    static final double PI = 3.14159;  // Universal constant

    static int max(int a, int b) {    // Utility method, no instance needed
        return a > b ? a : b;
    }
}

// Use INSTANCE when:
// 1. Each object needs its own copy
class Person {
    String name;  // Each person has their own name
    int age;      // Each person has their own age
}
```

Static Constants vs Instance Constants

```
class Circle {
    // Static constant - shared by all Circle objects
    static final double PI = 3.14159;

    // Instance field - each Circle has its own radius
    private double radius;

    public Circle(double radius) {
        this.radius = radius;
    }

    public double area() {
        // Can use static PI here
        return PI * radius * radius;  // PI is shared constant
    }

    public double circumference() {
        return 2 * PI * radius;
    }
}

Circle c1 = new Circle(5);
Circle c2 = new Circle(10);

c1.area();   // Uses PI=3.14159, radius=5
c2.area();   // Uses PI=3.14159, radius=10
// Both use SAME PI value (one copy in memory)
```

Edge Cases & Traps

Trap 1: Static Method Can't Access Instance Members

```
class Example {
    int x = 10;  // Instance field

    static void printX() {
        System.out.println(x);  // ERROR! Can't access instance field!
        // Because which x? There's no object context!
    }
}
```

[WARNING] **Professor Trap**: Static methods don't have `this` reference!

Trap 2: Static vs Instance Method Confusion

```
class Example {
    static int count = 0;
```

```
    int instanceCount = 0;

    static void incrementStatic() {
        count++;  // OK - static field
    }

    void incrementInstance() {
        instanceCount++;  // OK - instance field
        count++;          // OK - static field also accessible
    }
}
```

[WARNING] **Professor Trap**: Static can access static, instance can access both!

Trap 3: Modifying Static from Instance Method

```
class Counter {
    static int count = 0;

    Counter() {
        count++;  // All instances share same count!
    }
}

Counter c1 = new Counter();  // count = 1
Counter c2 = new Counter();  // count = 2
Counter c3 = new Counter();  // count = 3
// All three objects see same count = 3!
```

[WARNING] **Professor Trap**: Static fields are shared! All instances see same value!

Trap 4: New vs Old Style Constants

```
// Java pre-5 style (still works but outdated)
class Constants {
    static final int MAX_SIZE = 100;  // Uppercase with underscore
}

// Modern style: enum for related constants
enum Size { SMALL, MEDIUM, LARGE }

// Or: interface with constants (bad practice, but common)
interface Constants {
    int MAX_SIZE = 100;  // Automatically static final!
}
```

2.3 Constructors

Definition / Theoretical Background

A **constructor** is a special method that: 1. Has the **same name** as the class 2. Has **no return type** 3. Is called when creating new objects with `new` 4. Initializes the object's fields

Types of Constructors

```
class Dog {
    String name;

    // Implicit default constructor (if no other constructor defined)
    // Dog() { }  // Provided automatically if you define nothing

    // Explicit default constructor
    Dog() {
        name = "Unknown";
    }
}

Dog d = new Dog();  // Calls Dog()
System.out.println(d.name);  // "Unknown"
```

Default Constructor (No-Args)

```
class Dog {
    String name;
    int age;

    Dog(String name, int age) {
        this.name = name;   // Initialize name
        this.age = age;     // Initialize age
    }
}

Dog buddy = new Dog("Buddy", 3);  // Calls Dog(String, int)
```

Parameterized Constructor

```
class Student {
    String name;
    int age;
    String major;

    // Constructor 1: Full
```

```
    Student(String name, int age, String major) {
        this.name = name;
        this.age = age;
        this.major = major;
    }

    // Constructor 2: Two params
    Student(String name, int age) {
        this.name = name;
        this.age = age;
        this.major = "Undeclared";
    }

    // Constructor 3: One param
    Student(String name) {
        this.name = name;
        this.age = 0;
        this.major = "Undeclared";
    }

    // Constructor 4: No params
    Student() {
        this.name = "Unknown";
        this.age = 0;
        this.major = "Undeclared";
    }
}

// Usage
Student s1 = new Student("Alice", 20, "CS");
Student s2 = new Student("Bob", 19);
Student s3 = new Student("Charlie");
Student s4 = new Student();
```

Multiple Constructors (Constructor Overloading)

Constructor Chaining with `this()`

```
class Student {
    String name;
    int age;
    String major;

    // This constructor calls the full one
    Student(String name, int age) {
        this(name, age, "Undeclared");  // Chain to 3-param constructor
    }
```

```
    // This constructor calls the 2-param one
    Student(String name) {
        this(name, 0);  // Chain to 2-param constructor
    }

    // Full constructor (does actual work)
    Student(String name, int age, String major) {
        this.name = name;
        this.age = age;
        this.major = major;
    }
}
```

Constructor vs Setter

```
// Constructor: Set once, at creation
class ImmutableStudent {
    private final String name;
    private final int age;

    ImmutableStudent(String name, int age) {
        this.name = name;
        this.age = age;
        // No setters! Values never change after creation
    }
}

// vs

// Setter: Can change at any time
class MutableStudent {
    private String name;
    private int age;

    public void setName(String name) { this.name = name; }
    public void setAge(int age) { this.age = age; }
}

// When to use which?
// - Constructor: Values known at creation, won't change (immutable)
// - Setter: Values may change during object lifetime (mutable)
```

Private Constructor (Singleton Pattern)

```
class Singleton {
    private static Singleton instance;
```

```
    private String data;

    // Private constructor - can't call from outside!
    private Singleton() {
        data = "Important data";
    }

    // Only way to get instance
    public static Singleton getInstance() {
        if (instance == null) {
            instance = new Singleton();  // Create only once!
        }
        return instance;
    }
}

// Usage
Singleton s1 = Singleton.getInstance();
Singleton s2 = Singleton.getInstance();
System.out.println(s1 == s2);  // TRUE! Same object!
```

Edge Cases & Traps

Trap 1: Forgetting to Initialize Fields

```
class Student {
    int age;  // Default is 0, not uninitialized!

    // Student() { }  // Default constructor, age = 0
}

// Student s = new Student();
// s.age is 0, not garbage!
```

[WARNING] **Professor Trap**: Default constructor gives default values, not garbage!

Trap 2: Constructor Throwing Exception

```
class Bad {
    Resource resource;

    Bad() throws Exception {
        resource = new Resource();
        throw new Exception("Error!");  // Resource leaked!
    }
}

// If constructor throws, object is never fully created
// Destructor won't run!
```

```
// FIX: Use try-finally or RAII
```

Trap 3: Calling Overridable Method in Constructor

```
class Parent {
    Parent() {
        doSomething();  // BAD! Could call overridden method!
    }

    void doSomething() {
        System.out.println("Parent");
    }
}

class Child extends Parent {
    int x = 5;

    @Override
    void doSomething() {
        System.out.println("Child: " + x);  // x is 0 here! Not initialized yet!
    }
}

new Child();  // Prints "Child: 0" because child constructor hasn't run yet!
```

[WARNING] **Professor Trap**: Don't call overridable methods in constructors!

2.4 Information Hiding and Visibility Modifiers

The Four Access Levels

Modifier	Same Class	Same Package	Subclass	Anywhere
private	YES	NO	NO	NO
default (package-private)	YES	YES	NO	NO
protected	YES	YES	YES	NO
public	YES	YES	YES	YES

```
+-------------------------------------------------------------+
|                                                             |
|   public -------- Accessible EVERYWHERE                      |
|                                                             |
|   protected ------ Accessible within package AND subclasses   |
|                                                             |
|   (default) ------ Accessible within package only            |
|                                                             |
|   private ------- Accessible within class ONLY               |
|                                                             |
```

```
+------------------------------------------------------------+
```

Java Access Modifiers

```java
// public - accessible everywhere
public class PublicClass {
    public String name = "Everyone can see";

    public void publicMethod() { }  // Accessible by anyone
}

// protected - accessible in package + subclasses
class PackageClass {
    protected String name = "Package + subclasses";
    protected void protectedMethod() { }  // Accessible by subclass, package
}

// default (no modifier) - package-private
class DefaultClass {
    String name = "Only in my package";  // No modifier = package-private
    void defaultMethod() { }  // Only same package
}

// private - accessible in class only
class PrivateClass {
    private String secret = "Only this class";  // Hidden!
    private void privateMethod() { }  // Only this class

    // But private can be accessed by public methods of same class:
    public String getSecret() {
        return secret;  // OK! Same class
    }
}
```

Access Modifiers Visual

```
+------------------------------------------------------------+
| class MyClass {                                            |
|                                                           |
|     private int a = 1;    <--- Only MyClass can see       |
|     int b = 2;          <--- MyClass + same package       |
|     protected int c = 3; <--- MyClass + package + subclass |
|     public int d = 4;     <--- EVERYONE                    |
|                                                           |
|     public void method() {                                |
|         System.out.println(a);  // OK - same class        |
|         System.out.println(b);  // OK - same package      |
|         System.out.println(c);  // OK - same package      |
```

```
|         System.out.println(d);  // OK - everywhere          |
|     }                                                       |
| }                                                            |
+-------------------------------------------------------------+
```

Why Private is Protective

```
class BankAccount {
    private double balance;  // Only accessible within this class!

    public void deposit(double amount) {
        if (amount <= 0) {
            throw new IllegalArgumentException("Invalid amount");
        }
        balance += amount;  // OK - within class
    }

    public double getBalance() {
        return balance;  // Controlled access!
    }
}

// Outside code CANNOT do:
// account.balance = -1000;  // ERROR! Private!

// Must use:
// account.deposit(-1000);  // But deposit validates!
```

Python Naming Conventions (Analogous to Access Control)

```
class Person:
    def __init__(self, name):
        self.public_name = name     # public - accessible everywhere
        self._protected_name = name # _protected - convention only
        self.__private_name = name   # __private - name mangled

p = Person("Alice")

print(p.public_name)       # Works
print(p._protected_name)   # Works but convention says don't touch
# print(p.__private_name)   # ERROR! Name mangled to _Person__private_name
```

C++ Access Modifiers

```
class Example {
public:          // Accessible everywhere
    int publicField;
```

```
protected:        // Accessible in class and subclasses
    int protectedField;

private:          // Accessible in class only
    int privateField;
};
```

JavaScript Private Fields (ES2022+)

```
class Circle {
    // Private field (ES2022)
    #radius = 0;

    constructor(radius) {
        this.#radius = radius;  // Accessible within class
    }

    getRadius() {
        return this.#radius;  // OK - within class
    }
}

const c = new Circle(5);
console.log(c.radius);  // undefined (not accessible!)
console.log(c.getRadius());  // 5
```

What Should Be Private?

RULE: Make fields private, methods public (usually)

```
What to make PRIVATE:
- Fields (instance variables)
- Helper methods used only internally
- Complex logic that shouldn't be exposed
- Data that could corrupt state if changed directly

What to make PUBLIC:
- Methods that are the "contract" / API
- Getters for read-only access
- Setters only when modification is safe

What to make PROTECTED:
- Methods/fields subclasses might need
- Extension points for subclasses
```

Getter and Setter Pattern

```
class Person {
    private String name;   // PRIVATE - hidden
    private int age;        // PRIVATE - hidden

    // Public getter - READ access
    public String getName() {
        return name;
    }

    public int getAge() {
        return age;
    }

    // Public setter - CONTROLLED write access
    public void setName(String name) {
        if (name == null || name.isEmpty()) {
            throw new IllegalArgumentException("Invalid name");
        }
        this.name = name;
    }

    public void setAge(int age) {
        if (age < 0 || age > 150) {
            throw new IllegalArgumentException("Invalid age");
        }
        this.age = age;
    }
}

// Usage
Person p = new Person();
p.setName("Alice");  // Validated!
p.setAge(25);         // Validated!
// p.name = "Bob";   // ERROR! Private!
```

JavaScript Getters/Setters

```
class Person {
    #name;  // Private field

    constructor(name) {
        this.#name = name;
    }

    // Getter
```

```
    get name() {
        return this.#name;
    }

    // Setter
    set name(value) {
        if (!value) throw new Error("Name required");
        this.#name = value;
    }
}

const p = new Person("Alice");
console.log(p.name);  // Alice (calls getter)
p.name = "Bob";       // (calls setter)
```

Edge Cases & Traps

Trap 1: Overly Public Fields

```
// BAD! Exposing internal state!
class Point {
    public int x;  // Anyone can modify directly!
    public int y;
}

Point p = new Point();
p.x = -999999;  // Invalid! No protection!
```

[WARNING] **Professor Trap**: Always use private fields with getters/setters!

Trap 2: Setter That Does Nothing

```
// Useless setter!
public void setName(String name) {
    this.name = name;  // No validation = not better than public field!
}

// Better:
public void setName(String name) {
    if (name == null) throw new IllegalArgumentException();
    this.name = name;
}
```

Trap 3: Getter Returning Mutable Object

```
class Container {
    private List<String> items = new ArrayList<>();

    // DANGER! Returns reference to internal mutable object!
    public List<String> getItems() {
```

```
        return items;  // External code can modify!
    }
}

// External code:
container.getItems().add("HACKED!");  // Modifies internal list!

// FIX: Return copy
public List<String> getItems() {
    return new ArrayList<>(items);  // Return a copy!
}
```

[WARNING] **Professor Trap**: Don't return mutable internals! Return copies!

Trap 4: Public Static Field

```
// BAD! Shared mutable state!
public static List<String> sharedList = new ArrayList<>();

// FIX: Private static with accessor
private static final List<String> sharedList = new ArrayList<>();

public static List<String> getSharedList() {
    return Collections.unmodifiableList(sharedList);  // Read-only!
}
```

Examples

```
class BankAccount {
    // Private fields - hidden from outside
    private String accountNumber;
    private double balance;
    private Transaction[] transactions;
    private int transactionCount;

    // Constants
    private static final double OVERDRAFT_FEE = 35.0;
    private static final int MAX_TRANSACTIONS = 100;

    // Constructor
    public BankAccount(String accountNumber, double initialBalance) {
        if (accountNumber == null || accountNumber.isEmpty()) {
            throw new IllegalArgumentException("Invalid account number");
        }
        if (initialBalance < 0) {
            throw new IllegalArgumentException("Initial balance cannot be negative");
        }
        this.accountNumber = accountNumber;
```

```
        this.balance = initialBalance;
        this.transactions = new Transaction[MAX_TRANSACTIONS];
        this.transactionCount = 0;
    }

    // Public interface (API)
    public String getAccountNumber() {
        return accountNumber;  // Read-only, no setter
    }

    public double getBalance() {
        return balance;
    }

    public void deposit(double amount) {
        if (amount <= 0) {
            throw new IllegalArgumentException("Deposit amount must be positive");
        }
        balance += amount;
        addTransaction("DEPOSIT", amount);
    }

    public void withdraw(double amount) {
        if (amount <= 0) {
            throw new IllegalArgumentException("Withdrawal amount must be positive");
        }
        if (amount > balance) {
            balance -= OVERDRAFT_FEE;
            addTransaction("OVERDRAFT", OVERDRAFT_FEE);
            throw new IllegalStateException("Insufficient funds, overdraft fee charged");
        }
        balance -= amount;
        addTransaction("WITHDRAWAL", amount);
    }

    // Private helper - internal implementation hidden
    private void addTransaction(String type, double amount) {
        if (transactionCount < MAX_TRANSACTIONS) {
            transactions[transactionCount++] = new Transaction(type, amount);
        }
    }
}
```

Example 1: Fully Encapsulated Class

```
class Student {
    private String name;
    private int age;

    // Private constructor - can't use new Student() directly
    private Student(String name, int age) {
        this.name = name;
        this.age = age;
    }

    // Static factory methods - create instances with validation
    public static Student createFreshman(String name) {
        return new Student(name, 18);
    }

    public static Student createAdult(String name, int age) {
        if (age < 18) {
            throw new IllegalArgumentException("Adult must be 18+");
        }
        return new Student(name, age);
    }
}

// Usage
Student s1 = Student.createFreshman("Alice");  // age=18
Student s2 = Student.createAdult("Bob", 25);    // age=25
// Student s3 = new Student("Charlie", 20);   // ERROR! Private constructor!
```

Example 2: Static Factory Method

```
// In package com.example.models

class InternalData {
    String sensitiveInfo;  // Default (package-private)

    void internalMethod() { }  // Package-private method
}

// This class in same package can access:
class DataProcessor {
    void process() {
        InternalData data = new InternalData();
        data.sensitiveInfo = "Accessible";  // OK - same package
        data.internalMethod();  // OK - same package
    }
}
```

```
// But class in different package CANNOT:
class ExternalProcessor {
    void process() {
        InternalData data = new InternalData();
        data.sensitiveInfo = "NOT Accessible";  // ERROR!
        data.internalMethod();  // ERROR!
    }
}
```

Example 3: Package-Level Access

Practice Questions

MCQ 1: Which modifier makes a member accessible only within its class? - A) public - B) protected - C) private - D) default

MCQ 2: Can a static method access an instance variable? - A) Yes, directly - B) Only through an object reference - C) No, never - D) Only if it's final

MCQ 3: What is encapsulation? - A) Hiding implementation details - B) Making fields public - C) Creating multiple constructors - D) Inheriting from a class

Short Answer: Why should fields usually be private? What are the benefits?

Board Coding: Create a class `Temperature` with a private `celsius` field, public `getCelsius()`, `getFahrenheit()`, `setCelsius()`, and `setFahrenheit()` methods. Ensure all values are validated.

Oral Explanation: Explain the difference between public, protected, and private. Give an example of when you would use each.

Solutions

MCQ 1: C) private

MCQ 2: C) No, never - static methods don't have `this` reference

MCQ 3: A) Hiding implementation details

Short Answer: Private fields protect internal state from external corruption. All access goes through controlled public methods (getters/setters) that can validate input, ensuring the object always remains in a valid state. This is the "protective barrier" of encapsulation.

Board Coding:

```
class Temperature {
    private double celsius;

    public Temperature(double celsius) {
        setCelsius(celsius);
    }

    public double getCelsius() {
        return celsius;
```

```
    }

    public double getFahrenheit() {
        return celsius * 9/5 + 32;
    }

    public void setCelsius(double celsius) {
        if (celsius < -273.15) {
            throw new IllegalArgumentException("Below absolute zero!");
        }
        this.celsius = celsius;
    }

    public void setFahrenheit(double fahrenheit) {
        setCelsius((fahrenheit - 32) * 5/9);
    }
}
```

Cheat-Sheet

Concept	Key Points
Encapsulation	Bundle data + methods, hide implementation
Private	Accessible within class only
Protected	Accessible in class + package + subclasses
Public	Accessible everywhere
Default	Accessible in package only
Instance member	Each object has its own copy
Static member	One copy shared by all instances
Constructor	Initializes new objects
this()	Constructor chaining (must be first statement)

Review Checklist

- ☐ **Encapsulation**: Can you explain why it's important?
- ☐ **Access modifiers**: Do you know when to use each?
- ☐ **Static vs instance**: Can you explain the difference?
- ☐ **Private fields**: Do you know why to use them?
- ☐ **Getters/setters**: Can you implement proper validation?
- ☐ **Constructor types**: Default, parameterized, overloaded?
- ☐ **this() chaining**: Can you chain constructors properly?
- ☐ **Common traps**: Static accessing instance? Returning mutable? Calling overridable method in constructor? # Chapter 3: Polymorphism and Type Systems

3.1 Overloading

Definition / Theoretical Background

Overloading (also called **ad-hoc polymorphism**) is having multiple methods with the **same name** but **different parameters** within the same class or scope.

Two Types of Overloading:

Type	What Differs	Same?
Method Overloading	Number, type, or order of parameters	Return type can differ
Operator Overloading	Operand types (in some languages)	Different semantics

Method Overloading in Java

```
class Calculator {
    // Overload by number of parameters
    int add(int a, int b) {
        return a + b;
    }

    int add(int a, int b, int c) {
        return a + b + c;
    }

    // Overload by type of parameters
    double add(double a, double b) {
        return a + b;
    }

    // Overload by order (different types)
    String add(String a, int b) {
        return a + b;
    }

    int add(int a, String b) {
        return Integer.parseInt(b) + a;
    }
}

// Usage
Calculator calc = new Calculator();
calc.add(1, 2);          // Calls add(int, int)
calc.add(1, 2, 3);       // Calls add(int, int, int)
calc.add(1.5, 2.5);      // Calls add(double, double)
calc.add("Hello", 5);    // Calls add(String, int)
```

Method Overloading Rules

```
class Rules {
    // CAN overload (different parameters)
    void method(int x) { }
    void method(double x) { }
    void method(int x, int y) { }

    // CANNOT overload (only return type differs)
    // int method(int x) { }      // ERROR! Same signature as first!
    // double method(int x) { }    // ERROR!

    // CAN overload static vs instance (different context)
    static void method(int x) { }  // This is OK!
    void method(int x) { }          // Different because of static context
}
```

Signature Rule: Methods are distinguished by their **signature** (name + parameter types), NOT return type!

Constructor Overloading

```
class User {
    private String name;
    private int age;
    private String email;

    // Overloaded constructors
    User(String name) {
        this.name = name;
        this.age = 0;
        this.email = "not provided";
    }

    User(String name, int age) {
        this.name = name;
        this.age = age;
        this.email = "not provided";
    }

    User(String name, int age, String email) {
        this.name = name;
        this.age = age;
        this.email = email;
    }
}
```

Python Function Overloading (Different Approach)

```python
# Python doesn't have true overloading, but uses default arguments
class Calculator:
    def add(self, a, b=0, c=0):  # Flexible signature
        return a + b + c

calc = Calculator()
calc.add(1, 2)        # a=1, b=2, c=0 -> 3
calc.add(1, 2, 3)     # a=1, b=2, c=3 -> 6
```

C++ Operator Overloading

```cpp
class Fraction {
public:
    int numerator;
    int denominator;

    Fraction(int n, int d) : numerator(n), denominator(d) {}

    // Operator overloading
    Fraction operator+(const Fraction& other) {
        int n = numerator * other.denominator + other.numerator * denominator;
        int d = denominator * other.denominator;
        return Fraction(n, d);
    }
};

Fraction f1(1, 2);  // 1/2
Fraction f2(1, 3);  // 1/3
Fraction f3 = f1 + f2;  // Calls operator+
```

Edge Cases & Traps

Trap 1: Autoboxing/Unboxing Confusion

```java
class Example {
    void process(int x) { System.out.println("int: " + x); }
    void process(Integer x) { System.out.println("Integer: " + x); }
}

Example e = new Example();
e.process(5);      // Calls process(int) - primitive!
e.process(Integer.valueOf(5));  // Calls process(Integer) - boxed!

// With autoboxing, both might match!
```

[WARNING] **Professor Trap**: Primitive and boxed types can cause ambiguity!

Trap 2: Widening vs Boxing

```
class Confusing {
    void method(long x) { System.out.println("long"); }
    void method(Integer x) { System.out.println("Integer"); }
}

Confusing c = new Confusing();
c.method(5);  // Which one? long or Integer?
// Answer: long - intwidens to long, but int doesn't box to Integer for overload resolution!
```

[WARNING] **Professor Trap**: Primitive widening beats boxing! int -> long -> boxing to Integer is the order.

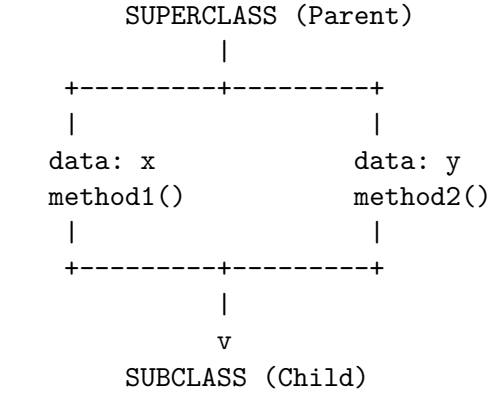
Trap 3: Varargs Last

```
class VarargsExample {
    void method(int x) { System.out.println("int"); }
    void method(int... args) { System.out.println("varargs"); }
}

VarargsExample v = new VarargsExample();
v.method(5);     // Calls method(int), not varargs! Exact match wins
```

3.2 Inheritance

Definition / Theoretical Background

Inheritance allows a class (subclass/derived) to inherit fields and methods from another class (superclass/base).




```
Inherits data: x, data: y, method1(), method2()
+ Can add NEW fields and methods
+ Can OVERRIDE existing methods
```

Inheritance Hierarchy in Java

```
// Superclass
class Animal {
    protected String name;
    protected int age;

    public Animal(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public void eat() {
        System.out.println(name + " is eating");
    }

    public void sleep() {
        System.out.println(name + " is sleeping");
    }
}

// Subclass - inherits from Animal
class Dog extends Animal {
    private String breed;

    // Subclass constructor
    public Dog(String name, int age, String breed) {
        super(name, age);  // Call parent constructor FIRST
        this.breed = breed;
    }

    // Subclass-specific method
    public void bark() {
        System.out.println(name + " says Woof!");
    }

    // Override parent's method
    @Override
    public void eat() {
        System.out.println(name + " (dog) is eating kibble");
    }
}

// Usage
Dog buddy = new Dog("Buddy", 3, "Golden Retriever");
buddy.eat();     // "Buddy (dog) is eating kibble" - overridden!
buddy.sleep();    // "Buddy is sleeping" - inherited!
```

```
buddy.bark();      // "Buddy says Woof!" - subclass only!
```

Inheritance Memory Visualization

```
When Dog inherits from Animal:

Animal fields:      Dog fields:
+--------------+    +--------------+
| name         |    | (inherited)  |
| age          |    | breed        |
+--------------+    +--------------+

Dog object in memory:
+--------------------------------+
|  Animal part (inherited)       |
|  +--------------+              |
|  | name         |              |
|  | age          |              |
|  +--------------+              |
|  Dog part (own)                |
|  +--------------+              |
|  | breed        |              |
|  +--------------+              |
+--------------------------------+
```

Single vs Multiple Inheritance

Type	Description	Languages
Single	One parent class	Java, C#, Python
Multiple	Multiple parent classes	C++ (yes), Python (yes), Java (no!)

```
// Java can ONLY extend ONE class
class A { }
class B { }
class C extends A, B { }  // ERROR! Java doesn't support multiple inheritance!
```

Java Single Inheritance

```
class A { void methodA() { } };
class B { void methodB() { } };

// C++ can extend multiple classes
class C : public A, public B {
    void methodC() {
```

```
        methodA();  // From A
        methodB();  // From B
    }
};
```

C++ Multiple Inheritance

```
class A:
    def method(self): print("A")

class B:
    def method(self): print("B")

class C(A, B):  # Multiple inheritance
    pass

c = C()
c.method()  # Which method? Prints "A" - first in MRO!

# MRO for C: C -> A -> B -> object
# Python uses left-to-right depth-first MRO
```

Python Multiple Inheritance (Method Resolution Order)

The super Keyword

```
class Parent {
    int x = 10;
    void display() { System.out.println("Parent: " + x); }
}

class Child extends Parent {
    int x = 20;

    void display() {
        System.out.println("Child x = " + x);       // x = 20 (own)
        System.out.println("Parent x = " + super.x);  // x = 10 (parent's)
    }

    @Override
    void display() {
        super.display();  // Call parent's display() first
        System.out.println("Child: " + x);
    }
}
```

Inheritance vs Composition

Inheritance ("is-a")	Composition ("has-a")
Dog **IS-AN** Animal	Car **HAS-A** Engine
Student **IS-A** Person	Library **HAS-A** Books
Circle **IS-A** Shape	Team **HAS-A** Members

```
// INHERITANCE - Use when "is-a" relationship
class Animal { void eat() { } }
class Dog extends Animal { void bark() { } }

// COMPOSITION - Use when "has-a" relationship
class Engine { void start() { } }
class Car {
    private Engine engine;  // Car HAS an Engine
    void start() { engine.start(); }
}
```

When to Use Which: - **Inheritance**: When subclass truly IS-A superclass (Dog is Animal) - **Composition**: When class HAS-A relationship with another class (Car has Engine)

[WARNING] **Favor composition over inheritance** - Inheritance creates tight coupling!

Edge Cases & Traps

Trap 1: Private Members Are Not Inherited

```
class Parent {
    private int secret = 42;  // PRIVATE - NOT inherited!
    public int getSecret() { return secret; }  // Must use getter
}

class Child extends Parent {
    // secret is NOT visible here!
    // System.out.println(secret);  // ERROR!
    // But can access via:
    System.out.println(getSecret());  // Works!
}
```

Trap 2: Single Inheritance Only in Java

```
class A { }
class B { }
class C { }
// Java: class D extends A, B, C { }  // COMPILE ERROR!
// Java only allows ONE parent class!
```

Trap 3: Diamond Problem (Multiple Inheritance Ambiguity)

```cpp
// C++ handles this, but can be confusing
class A { void method() { std::cout << "A"; } };
class B { void method() { std::cout << "B"; } };

class C : public A, public B {
    void test() {
        // method();  // AMBIGUOUS! Which method()?
        A::method();  // Explicitly call A's method
        B::method();  // Explicitly call B's method
    }
};
```

Trap 4: Overriding vs Overloading Confusion

```java
class Parent {
    void process(int x) { System.out.println("Parent int"); }
}

class Child extends Parent {
    void process(double x) { System.out.println("Child double"); }
}

Child c = new Child();
c.process(5);  // Calls process(double) because 5 is widened to 5.0!
c.process(5.0);  // Calls process(double)

Parent p = c;  // UPCAST
p.process(5);  // Calls Parent's process(int)! Polymorphism!
```

3.3 Subtyping, Static and Dynamic Types, Type Checking

Subtyping

Subtype = A type that can be used where a supertype is expected.

```java
class Animal { }
class Dog extends Animal { }  // Dog is subtype of Animal

Animal a = new Dog();  // Valid! Dog can be used as Animal
// a is statically Animal, but dynamically Dog
```

The Liskov Substitution Principle: > "Objects of a superclass should be replaceable with objects of its subclasses without breaking the application."

```java
// If Dog extends Animal, then anywhere Animal is expected, Dog can be used!
void feedAnimal(Animal a) { }  // Accepts Dog, Cat, Bird, etc.

Dog d = new Dog();
```

```
feedAnimal(d);  // Works! Dog IS-A Animal
```

Static vs Dynamic (Runtime) Type

```
class Animal { void speak() { System.out.println("..."); } }
class Dog extends Animal { @Override void speak() { System.out.println("Woof"); } }
class Cat extends Animal { @Override void speak() { System.out.println("Meow"); } }

Animal a = new Dog();
// Static type: Animal (known at compile time)
// Dynamic type: Dog (known only at runtime)

a = new Cat();
// Static type: Animal (compiler sees Animal)
// Dynamic type: Cat (assigned at runtime)

// Compiler only knows static type - decides what methods are callable
// a.speak();  // Valid! speak() exists in static type
// a.bark();   // ERROR! bark() not in static type (Animal)
```

Static vs Dynamic Type Visualization:

```
COMPILE TIME (Static Type)
+-------------------------------------+
|  Animal a;                          |
|  Compiler only knows: a is Animal   |
|  Allowed calls: speak(), eat(), etc. |
|  NOT allowed: bark(), meow()        |
+-------------------------------------+
         |
         v
RUNTIME (Dynamic Type)
+-------------------------------------+
|  a = new Dog();  // or new Cat()    |
|  At runtime, we KNOW actual type    |
|  Dog.dogBark() or Cat.meow() called |
+-------------------------------------+
```

Type Checking

```
// Type checked at COMPILE TIME
Animal a = new Dog();  // OK - Dog is-a Animal
a.bark();              // ERROR - bark() not in Animal
```

Static Typing (Java, C++, C#)

```
# Type checked at RUNTIME
a = Dog()  # No type declaration
a.bark()   # Works at runtime if Dog has bark(), error otherwise
```

Dynamic Typing (Python, JavaScript)

```
# "If it walks like a duck and quacks like a duck..."
# No type inheritance needed!
class Duck:
    def quack(self): print("Quack!")

class Person:
    def quack(self): print("I'm pretending!")

def make_it_quack(obj):
    obj.quack()  # Any object with quack() works!

make_it_quack(Duck())    # Works
make_it_quack(Person())  # Works too!
```

Duck Typing (Python)

Type Casting / Casting

```
class Animal { void eat() { } }
class Dog extends Animal { void bark() { } }
class Cat extends Animal { void meow() { } }

// Upcast (safe, implicit)
Dog d = new Dog();
Animal a = d;  // Upcast to Animal - always safe!

// Downcast (unsafe, explicit)
Animal a = new Dog();  // a is actually a Dog
Dog d2 = (Dog) a;      // Downcast - must be explicit!
d2.bark();              // Works because a is really a Dog

// Downcast without checking (DANGEROUS!)
Animal a2 = new Cat();  // a2 is actually a Cat
Dog d3 = (Dog) a2;      // Compiles, but RUNTIME ERROR at a3.bark()!
// ClassCastException: Cat cannot be cast to Dog
```

```
Animal a = new Cat();
```

```java
if (a instanceof Dog) {
    Dog d = (Dog) a;  // Safe to cast
    d.bark();
}
else if (a instanceof Cat) {
    Cat c = (Cat) a;  // Safe
    c.meow();
}

// Modern Java: pattern matching
if (a instanceof Dog d) {  // Java 16+
    d.bark();  // d is already cast to Dog
}
```

instanceof (Type Checking)

Edge Cases & Traps

Trap 1: Static vs Dynamic Method Dispatch

```java
class Parent {
    void method() { System.out.println("Parent"); }
    static void staticMethod() { System.out.println("Parent static"); }
}

class Child extends Parent {
    @Override
    void method() { System.out.println("Child"); }  // Instance method - overridden
    static void staticMethod() { System.out.println("Child static"); }  // HIDES, not override
}

Parent p = new Child();
p.method();        // "Child" - runtime dispatch (polymorphism!)
p.staticMethod();  // "Parent static" - static methods don't override!
```

[WARNING] **Professor Trap**: Static methods are hidden, not overridden!

Trap 2: Object vs Reference Confusion

```java
// What is printed?
class Base {
    int x = 5;
}

class Derived extends Base {
    int x = 10;
}

Derived d = new Derived();
System.out.println(d.x);        // 10 (Derived's x)
```

```
Base b = d;                          // Upcast
System.out.println(b.x);             // 5 (Base's x!)
// Field x is NOT overridden - it's SHADOWED!
// Use getters to ensure proper polymorphism
```

3.4 Overriding and Dynamic Binding

Method Overriding

Overriding = Providing a new implementation for an inherited method in the subclass.

```
class Animal {
    void makeSound() { System.out.println("Some sound"); }
}

class Dog extends Animal {
    @Override  // Annotation (optional but recommended)
    void makeSound() { System.out.println("Woof!"); }
}

class Cat extends Animal {
    @Override
    void makeSound() { System.out.println("Meow!"); }
}
```

Dynamic Binding (Polymorphism)

```
+-------------------------------------------------------------+
|                   DYNAMIC METHOD DISPATCH                   |
|                                                             |
|   Animal[] animals = { new Dog(), new Cat(), new Dog() };   |
|                                                             |
|   for (Animal a : animals) {                                |
|       a.makeSound();                                        |
|   }                                                         |
|                                                             |
|   At compile time:  a.makeSound() is called on Animal       |
|   At runtime:       Actual type determines which version runs |
|                                                             |
|   Output:                                                   |
|   "Woof!"   (Dog's makeSound)                               |
|   "Meow!"   (Cat's makeSound)                               |
|   "Woof!"   (Dog's makeSound)                               |
|                                                             |
|   SAME CODE - DIFFERENT behavior based on ACTUAL type!      |
+-------------------------------------------------------------+
```

The Override Annotation

```
class Parent {
    void display() { System.out.println("Parent"); }
}

class Child extends Parent {
    @Override  // Tells compiler to check this actually overrides something
    void display(int x) { System.out.println("Child"); }  // ERROR! No method to override!

    @Override  // This one is correct
    void display() { System.out.println("Child"); }
}
```

Overriding Rules

```
class Base {
    // CAN override
    void method() { }

    // CANNOT override
    private void method();          // Private - not inherited!
    final void method();           // Final - cannot override
    static void method();           // Static - hidden, not overridden
}
```

Rules Table:

Aspect	Override Rule
Name	Must be same
Parameters	Must be same (or compatible)
Return type	Must be same or covariant (subtype)
Access	Cannot be more restrictive
Exception	Cannot throw new unchecked exceptions

Covariant Return Type

```
class Animal { }
class Dog extends Animal { }

class Cage {
    Animal getAnimal() { return new Animal(); }
}

class DogCage extends Cage {
    @Override
```

```
    Dog getAnimal() { return new Dog(); }  // Covariant - Dog is subtype of Animal
}
```

Abstract Methods and Classes

```
// Abstract class - CAN'T be instantiated
abstract class Shape {
    abstract double area();  // Abstract - no implementation
    abstract double perimeter();

    // Can have concrete methods too
    void printInfo() {
        System.out.println("Area: " + area());
    }
}

class Circle extends Shape {
    double radius;

    Circle(double radius) { this.radius = radius; }

    @Override
    double area() { return Math.PI * radius * radius; }

    @Override
    double perimeter() { return 2 * Math.PI * radius; }
}

// Shape s = new Shape();  // ERROR - abstract!
Shape s = new Circle(5);  // OK - concrete subclass
s.area();  // Calls Circle's area!
```

3.5 Interfaces

Definition / Theoretical Background

An **interface** is a contract that defines a set of method signatures without implementation.

```
INTERFACE (Contract)                    IMPLEMENTATIONS
+---------------------+                +--------------+
| + fly()             | -------------- | Bird : Flyer  |
| + land()            |                +--------------+
| + takeoff()         |                +--------------+
+---------------------+                |Plane : Flyer |
                                        +--------------+
Contract: ANYTHING that implements Flyer
          must provide fly(), land(), takeoff()
```

Java Interface Example

```java
// Interface definition
interface Drawable {
    void draw();  // Abstract method (implicitly)

    // Java 8+: default method
    default void print() {
        System.out.println("Printing...");
    }

    // Java 8+: static method
    static void info() {
        System.out.println("Drawable interface");
    }
}

interface Serializable {
    void serialize();
}

// Class implementing multiple interfaces
class Circle implements Drawable, Serializable {
    double radius;

    Circle(double radius) { this.radius = radius; }

    @Override
    public void draw() {
        System.out.println("Drawing circle with radius " + radius);
    }

    @Override
    public void serialize() {
        // serialization logic
    }
}
```

Interface vs Abstract Class

Aspect	Interface	Abstract Class
Methods	Abstract (no implementation)	Can have both
Fields	public static final only	Any type
Constructors	None	Can have constructors
Multiple inheritance	Yes (implements multiple)	No (extends one)
Use when	"Can-do" relationship	"Is-a" relationship

Aspect	Interface	Abstract Class
State	No instance fields	Can have instance fields

```
// Interface - "can do" (capability)
interface Flyable {
    void fly();
}

interface Swimmable {
    void swim();
}

// Duck CAN do both!
class Duck implements Flyable, Swimmable {
    @Override
    public void fly() { System.out.println("Duck flying"); }

    @Override
    public void swim() { System.out.println("Duck swimming"); }
}
```

Interface as Type

```
interface Drawable {
    void draw();
}

class Circle implements Drawable { /* ... */ }
class Square implements Drawable { /* ... */ }

class Canvas {
    // Accepts ANY class that implements Drawable
    void addShape(Drawable d) {
        d.draw();  // Don't care what actual type is!
    }
}

Canvas canvas = new Canvas();
canvas.addShape(new Circle());   // OK
canvas.addShape(new Square());   // OK
```

Default Methods (Java 8)

```
interface Logger {
    void log(String message);  // Abstract
```

```
    // Default - provides implementation
    default void logError(String message) {
        log("[ERROR] " + message);
    }

    // Static - belongs to interface
    static Logger defaultLogger() {
        return System.out::println;
    }
}
```

Edge Cases & Traps

Trap 1: Interface vs Abstract Class Confusion

```
// When to use abstract class vs interface?
// Abstract class: When classes share state/implementation
abstract class Shape {
    protected String color;  // Shared state
    abstract double area();   // To implement
}

// Interface: When only contract needed
interface Drawable {
    void draw();  // Just the contract
}
```

Trap 2: Adding Methods to Interface (Breaking Change)

```
// Old interface
interface MyInterface {
    void oldMethod();
}

// Adding new method breaks ALL implementing classes!
interface MyInterface {
    void oldMethod();
    void newMethod();  // All implementations must add this!
}

// Use default method to avoid breaking:
interface MyInterface {
    void oldMethod();
    default void newMethod() { /* empty default */ }
}
```

Trap 3: Multiple Interface Conflict

```java
interface A { default void method() { System.out.println("A"); } }
interface B { default void method() { System.out.println("B"); } }

class C implements A, B {
    @Override
    public void method() {
        A.super.method();  // Must explicitly choose!
        // Or: B.super.method();
    }
}
```

3.6 Generics

Definition / Theoretical Background

Generics allow writing reusable code that works with different types while maintaining type safety at compile time.

```java
// WITHOUT generics - unchecked, unsafe
class Box {
    private Object item;
    public void set(Object item) { this.item = item; }
    public Object get() { return item; }
}

Box b = new Box();
b.set("hello");
String s = (String) b.get();  // Cast needed, can fail!

// WITH generics - type safe, checked at compile time
class Box<T> {  // T is type parameter
    private T item;
    public void set(T item) { this.item = item; }
    public T get() { return item; }  // No cast needed!
}

Box<String> b2 = new Box<>();
b2.set("hello");
String s2 = b2.get();  // No cast! Compiler knows it's String!
```

Generic Class

```java
class Pair<K, V> {
    private K key;
    private V value;
```

```
    public Pair(K key, V value) {
        this.key = key;
        this.value = value;
    }

    public K getKey() { return key; }
    public V getValue() { return value; }
}

Pair<String, Integer> p = new Pair<>("Age", 25);
String key = p.getKey();   // No cast!
Integer val = p.getValue(); // No cast!
```

Generic Methods

```
class Utils {
    // Generic method - type inferred from arguments
    public static <T> void print(T item) {
        System.out.println(item);
    }

    public static <T> int indexOf(T[] array, T item) {
        for (int i = 0; i < array.length; i++) {
            if (array[i].equals(item)) return i;
        }
        return -1;
    }
}

String[] names = {"Alice", "Bob"};
int pos = Utils.indexOf(names, "Bob");  // Type inferred
Utils.print("Hello");  // Type inferred as String
```

Bounded Type Parameters

```
// T must be a subclass of Number
class NumberBox<T extends Number> {
    private T value;

    public T getValue() { return value; }

    public double doubleValue() {
        return value.doubleValue();  // Can call Number methods!
    }
}
```

```
NumberBox<Integer> nb = new NumberBox<>(5);  // OK
NumberBox<Double> nd = new NumberBox<>(3.14);  // OK
// NumberBox<String> ns = new Box<>("hi");  // ERROR! String doesn't extend Number!
```

Upper and Lower Bounds

```
// Upper bound - T must be Number or subclass
void processNumbers(List<? extends Number> list) {
    // Can READ from list (as Number), cannot WRITE
    for (Number n : list) {
        System.out.println(n.doubleValue());
    }
    // list.add(5);  // ERROR! Don't know exact type
}

// Lower bound - T must be Integer or superclass
void processIntegers(List<? super Integer> list) {
    // Can WRITE to list, can READ as Object
    list.add(5);  // OK! List is at least Integer[]
    // Integer i = list.get(0);  // ERROR! Only know it's at least Object
}

// PECS: Producer Extends, Consumer Super
```

Generic Constraints

```
// Multiple bounds
class MyClass<T extends Comparable<T> & Serializable> {
    // T must implement both Comparable AND Serializable
}

// Interface vs Class in bounds
class A { }
interface B { }
class C<T extends A & B> { }  // T extends A AND implements B
```

Type Erasure

```
// At compile time, generics are erased
class Box<T> {
    private T item;
    public T get() { return item; }
}

// After erasure (at runtime):
class Box {
```

```
    private Object item;  // T becomes Object (or bound)
    public Object get() { return item; }
}

// This means:
Box<String> s = new Box<>();
Box b = s;  // Same class! Cannot do: b.set(5); // No type check at runtime
```

Edge Cases & Traps

Trap 1: Type Erasure at Runtime

```
List<String> strings = new ArrayList<>();
List<Integer> ints = new ArrayList<>();

// At RUNTIME, both are just List!
System.out.println(strings.getClass() == ints.getClass());  // TRUE!

// Cannot do generic type checking at runtime:
if (list instanceof List<String>) { }  // ERROR! Type erased!
if (list instanceof List<?>) { }       // OK! Use wildcard
```

Trap 2: Generic Arrays

```
// Cannot create generic array directly
// T[] array = new T[10];  // ERROR!
T[] array = (T[]) new Object[10];  // Works but unchecked cast warning

// Better approach:
class Container<T> {
    private Object[] elements;
    public Container(int size) {
        elements = new Object[size];
    }
    public T get(int index) {
        return (T) elements[index];  // Cast needed
    }
}
```

Trap 3: Primitive Types Don't Work

```
// List<int> list = new ArrayList<>();  // ERROR! No primitives!

// Must use wrapper classes:
List<Integer> list = new ArrayList<>();  // Integer, not int
list.add(5);  // Autoboxes to Integer(5)
int x = list.get(0);  // Unboxes to int
```

3.7 Subtype and Parametric Polymorphism

Types of Polymorphism

Type	Description	When
Ad-hoc (Overloading)	Same name, different params	Compile time
Parametric (Generics)	Code works with any type	Compile time
Subtype (Inheritance)	One interface, many implementations	Runtime

Subtype Polymorphism

```
// Same method, different behavior based on actual type
interface Shape {
    double area();
}

class Circle implements Shape {
    double radius;
    @Override
    double area() { return Math.PI * radius * radius; }
}

class Rectangle implements Shape {
    double width, height;
    @Override
    double area() { return width * height; }
}

void printArea(Shape s) {
    System.out.println(s.area());  // Calls correct area()!
}

printArea(new Circle(5));      // Prints circle area
printArea(new Rectangle(4, 5)); // Prints rectangle area
```

Parametric Polymorphism

```
// One implementation works for ANY type
class Stack<T> {
    private T[] elements;
    private int top;

    public void push(T item) { /* ... */ }
    public T pop() { /* ... */ return elements[top]; }
}

Stack<Integer> intStack = new Stack<>();  // For integers
```

```
Stack<String> strStack = new Stack<>();    // For strings
// Same Stack class works for both!
```

Combined Polymorphism Example

```
// Subtype + Generic = Powerful combination
interface Repository<T> {
    void save(T item);
    T findById(long id);
}

class User { }

class UserRepository implements Repository<User> {
    @Override
    public void save(User user) { /* DB save */ }

    @Override
    public User findById(long id) { /* DB query */ }
}

// Can treat many repositories uniformly
class Service<T> {
    private Repository<T> repository;

    public Service(Repository<T> repo) {
        this.repository = repo;
    }

    public void saveAll(List<T> items) {
        for (T item : items) {
            repository.save(item);
        }
    }
}
```

Practice Questions

MCQ 1: When is method overloading resolved? - A) Runtime - B) Compile time - C) Depends on language - D) Never

MCQ 2: What does `@Override` annotation do? - A) Changes method to static - B) Forces compile-time override check - C) Makes method public - D) Nothing

MCQ 3: What is type erasure in Java generics? - A) Removing type information at runtime - B) Adding type information at compile time - C) Converting primitives to objects - D) Runtime type checking

Short Answer: Explain the difference between static typing and dynamic typing.

Board Coding: Create an interface `Shape` with `area()` and `perimeter()` methods, then implement `Circle` and `Rectangle` classes.

Oral Explanation: Explain the difference between method overloading and method overriding. When would you use each?

Solutions

MCQ 1: B) Compile time - overloading resolved by parameter types at compile time

MCQ 2: B) Forces compile-time override check - compiler verifies method actually overrides a parent method

MCQ 3: A) Removing type information at runtime - generic type parameters are erased and replaced with Object or bounds

Short Answer: Static typing checks types at compile time (Java, C++), catching errors before runtime. Dynamic typing checks at runtime (Python, JavaScript), allowing more flexibility but delaying error detection.

Cheat-Sheet

Concept	Key Points
Overloading	Same name, different params, compile time
Overriding	Same signature, new implementation, runtime
Inheritance	is-a relationship, extends parent
Subtype polymorphism	Same interface, different implementation at runtime
Generics	Type parameters, , compile-time safety
Interface	Contract, implements keyword, multiple allowed
Abstract class	Cannot instantiate, can have partial implementation
Dynamic dispatch	Runtime method selection based on actual object type

Review Checklist

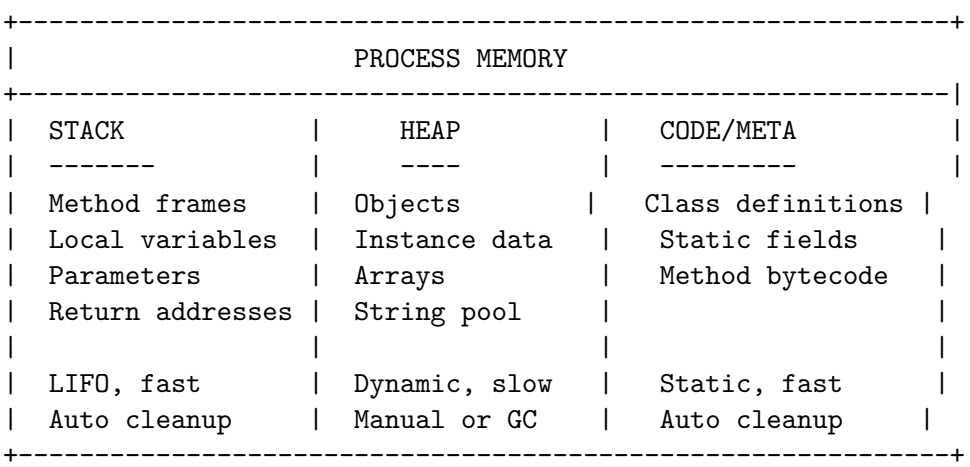
- ☐ **Overloading**: Can you identify overloaded methods?
- ☐ **Overriding**: Can you override methods correctly with rules?
- ☐ **Inheritance**: Can you explain is-a vs has-a relationship?
- ☐ **Static vs dynamic type**: Do you understand the difference?
- ☐ **Interfaces**: Can you design and implement interfaces?
- ☐ **Generics**: Can you use and create generic classes/methods?
- ☐ **Type erasure**: Do you understand Java's erasure mechanism?
- ☐ **Polymorphism**: Can you explain all three types of polymorphism? # Chapter 4: Memory Management

4.1 Memory Management Overview

Definition / Theoretical Background

Memory Management is the process of allocating, using, and releasing memory during program execution. In OOP languages, this involves managing:

- **Heap memory** for objects (instances)
- **Stack memory** for method calls and local variables
- **Static/Code memory** for class metadata



Memory in OOP Languages

Language	Object Storage	Garbage Collection
Java	Heap	Yes (automatic)
Python	Heap	Yes (automatic)
C#	Heap	Yes (automatic)
C++	Heap or Stack	No (manual)
Go	Heap	Yes (automatic, concurrent)
Rust	Heap (owned)	Yes (ownership model)

4.2 Stack and Heap in OOP

Stack Memory in Java/Python

```
class Example {
    int instanceVar;  // On heap with object

    void method(int param) {  // param on stack
        int local = 10;       // local on stack
        Object obj = new Object();  // Reference on stack, object on heap
    }
}
```

```
STACK (method calls)              HEAP (objects)
+---------------------+         +---------------------+
| method() frame:     |         | Object() instance   |
|   param = ...       |-------->|                     |
|   local = 10        |         |                     |
|   obj = ref --------+-------->|                     |
+---------------------+         +---------------------+
```

Object Allocation

```
class Dog {
    String name;
    int age;
}

// Creating objects
Dog buddy = new Dog();
```

What `new` does: 1. Allocates memory on heap (size of Dog class) 2. Initializes fields to default values (null, 0) 3. Calls constructor 4. Returns reference (address) to the object

```
new Dog() allocates on heap:
+--------------------------------------+
|            Dog object                |
|  +---------------+-----------------+ |
|  | String name   | null (default)  | |
|  | int age       | 0 (default)     | |
|  +---------------+-----------------+ |
|                                      |
|  Total: ~16 bytes depending on JVM   |
+--------------------------------------+
```

Memory Visualization

```
class Owner {
    String name;
    Dog dog;  // Reference to Dog
}

Owner o = new Owner();  // Owner object on heap
o.name = "Alice";
o.dog = new Dog();       // Dog object on heap
o.dog.name = "Buddy";   // Dog's name

// Memory looks like:
/*
HEAP:
+-------------------------+
```

```
| Owner:                    |
|   name --> "Alice"        |
|   dog --------------------+--+
+---------------------------+  |
                                |
+---------------------------+  |
| Dog:                       |  |
|   name --> "Buddy"        |<-+
|   age = 0                 |
+---------------------------+
*/
```

Stack vs Heap Comparison in OOP

Aspect	Stack	Heap
What stored	Primitives, references	Objects, arrays
Lifetime	Until method returns	Until GC reclaims
Access	LIFO (method order)	Random (via references)
Size	Small, fixed limit	Large, dynamic
Speed	Fast	Slower
Management	Automatic	Manual or GC

4.3 Garbage Collection

Definition / Theoretical Background

Garbage Collection (GC) is the automatic process of reclaiming memory that is no longer in use.

When is memory "garbage"? 1. Object has no references pointing to it 2. Reference chain doesn't reach the object

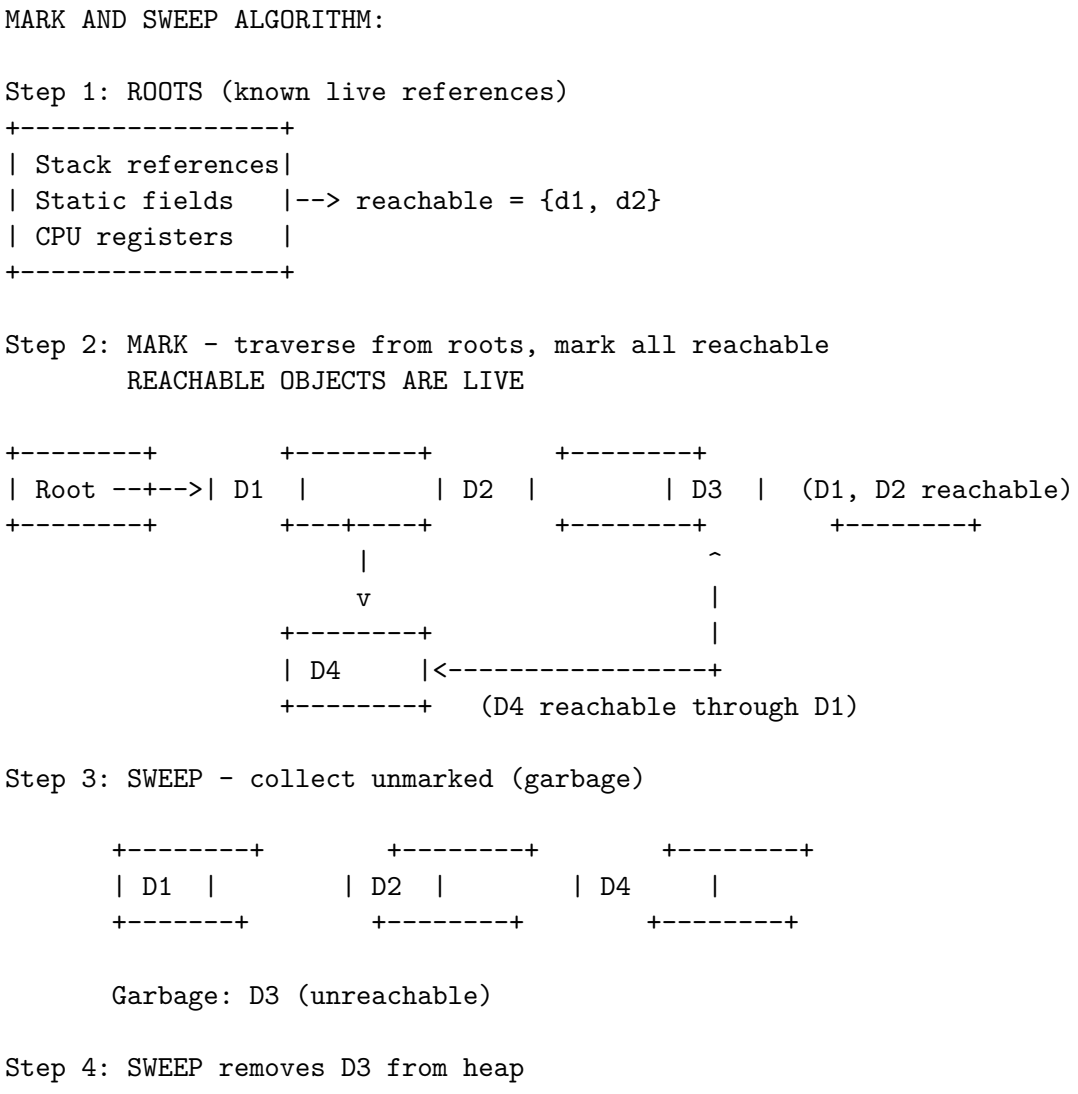
```
Dog d = new Dog();  // Object exists, referenced by d
d = null;           // Object now has NO references!
// Object becomes garbage - GC will reclaim it eventually
```

Object Eligibility for GC

```
// Three paths to garbage collection:

// 1. Reference goes out of scope
void method() {
    Dog d = new Dog();  // Object created
}  // d goes out of scope here, object eligible for GC

// 2. Reference reassigned
Dog d = new Dog("Buddy");
```

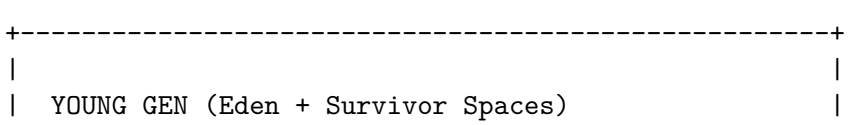
```
d = new Dog("Max");  // First Dog object now has no references!

// 3. Reference set to null
Dog d = new Dog();
d = null;  // Object now eligible for GC
```

GC Algorithm Visualization



Generational GC

Most modern GCs use **generational collection** - objects are divided by age:



```
|  +-- Eden: New objects allocated here               |
|  +-- S0: Surviving minor GC #1                      |
|  +-- S1: Surviving minor GC #2                       |
|                                                      |
|  OLD GEN (Tenured): Long-lived objects              |
|  Objects that survive many young GC cycles move here |
|                                                      |
|  METASPACE/Native: Class definitions, interned      |
|  strings (JVM specific)                              |
+------------------------------------------------------+

Hypothesis: Most objects die young (80-98%!)
=> Collect young generation frequently
=> Old generation collected rarely
```

GC Types in Different Languages

```
// JVM options for different collectors:
// -XX:+UseSerialGC      Serial (stop-the-world)
// -XX:+UseParallelGC   Parallel (throughput)
// -XX:+UseConcMarkSweepGC  CMS (low latency)
// -XX:+UseG1GC          G1 (default in Java 9+)
// -XX:+UseZGC           ZGC (ultra-low latency)
```

Java GC (Multiple Algorithms) G1 (Garbage First) - Default since Java 9: - Divides heap into regions - Tracks live objects per region - Collects regions with most garbage first - Predictable pause times

```
import gc

# Python uses reference counting (immediate) + cycle detector (periodic)
import sys

obj = object()
print(sys.getrefcount(obj))  # 2 (obj + parameter to getrefcount)

# Cycle detector finds reference cycles that counting misses:
gc.collect()  # Force cycle collection

# Can find cycles:
gc.garbage  # List of objects with reference cycles
```

Python GC (Reference Counting + Cycle Detector)

```
// .NET GC has generations: 0, 1, 2
// Gen0: Short-lived objects (ephemeral)
// Gen1: Buffer between short and long lived
// Gen2: Long-lived objects

// Explicit hints
GC.Collect();  // Force collection
GC.Collect(0);  // Gen0 only
GC.Collect(2, GCCollectionMode.Forced);

// Keep object alive across generations:
GC.KeepAlive(obj);  // obj cannot be collected before this point
```

C# GC

Memory Leaks in OOP

Memory leak = When programs allocate memory but fail to release it.

```
// Common leak: Collection that grows forever
class Cache {
    private static List<Data> cache = new ArrayList<>();

    public void addData(Data d) {
        cache.add(d);  // Keeps growing! Objects never removed!
    }
}

// Leak patterns:
// 1. Forgetting to remove from collections
// 2. Caching without eviction policy
// 3. Listeners/callbacks not unregistered
// 4. Static references to objects
// 5. Inner class holding outer class reference (in Java)
```

How to Avoid Memory Leaks

```
// 1. Use weak references for caches
import java.lang.ref.WeakHashMap;

// 2. Clear collections when done
cache.clear();

// 3. Use try-with-resources for cleanup
try (Connection conn = getConnection()) {
    // Use connection
} // Auto-closed!
```

```
// 4. Unregister listeners
button.removeActionListener(listener);

// 5. Use proper scope
void process() {
    // Local object - auto-cleaned when method ends
    TempData data = new TempData();
    // ...
} // data eligible for GC
```

4.4 Manual Memory Management (C++)

No Automatic GC - Manual Deletion

```
class Dog {
public:
    string name;
    Dog(const string& n) : name(n) {}
    ~Dog() { cout << name << " destroyed\n"; }
};

// Creating objects
Dog* d = new Dog("Buddy");  // Heap allocation
delete d;  // MUST delete manually!

// Stack allocation (automatic cleanup)
Dog d2("Max");  // Automatically destroyed when goes out of scope
```

RAII (Resource Acquisition Is Initialization)

```
// RAII - wrap resource in object destructor
class FileHandler {
    FILE* file;
public:
    FileHandler(const char* name) {
        file = fopen(name, "r");
    }
    ~FileHandler() {
        if (file) fclose(file);  // Automatic cleanup!
    }
};

void process() {
    FileHandler fh("data.txt");
    // Use file...
```

```
} // fh automatically closed when going out of scope!

// Compare to C-style:
void processC() {
    FILE* f = fopen("data.txt", "r");
    // ...
    fclose(f);  // Easy to forget! And exceptions can bypass this!
}
```

Smart Pointers (Modern C++)

```
#include <memory>

// Unique pointer - exclusive ownership
unique_ptr<Dog> dog1 = make_unique<Dog>("Buddy");
// dog1 automatically deleted when goes out of scope!

// Shared pointer - shared ownership
shared_ptr<Dog> dog2 = make_shared<Dog>("Max");
// Reference counted, deleted when last reference gone

void function() {
    shared_ptr<Dog> dog3 = dog2;  // Reference count becomes 2
} // dog3 destroyed, count back to 1

// Weak pointer - non-owning reference
weak_ptr<Dog> weakDog = dog2;  // Doesn't increase count
if (auto locked = weakDog.lock()) {
    // Only access if object still exists
}

// No delete needed with smart pointers!
```

Smart Pointer Comparison

Pointer	Ownership	Copy	Use When
`unique_ptr<T>`	Exclusive	No	Single owner, transfer with move
`shared_ptr<T>`	Shared	Yes (ref count)	Multiple owners
`weak_ptr<T>`	None	Yes	Break cycles, observe

Dangling Pointers

```
// Dangling pointer - points to deleted memory
Dog* createDog() {
    Dog d("Buddy");
```

```
    return &d;  // WRONG! d destroyed when function ends!
}

Dog* safeDog() {
    Dog* d = new Dog("Buddy");  // Heap allocation
    return d;  // OK - d won't be destroyed
}

// When done:
delete safeDog();  // Delete when finished

// Better: return smart pointer
shared_ptr<Dog> createDogSmart() {
    return make_shared<Dog>("Buddy");  // Auto-cleanup!
}
```

Memory Leak Example

```
void leak() {
    Dog* d = new Dog("Leaky");
    // ... some code ...
    if (error) return;  // Oops! Forgot delete!
    delete d;
}

// Fixed with unique_ptr:
void noLeak() {
    unique_ptr<Dog> d = make_unique<Dog>("Safe");
    if (error) return;  // No problem! d auto-deleted!
}
```

4.5 Object Lifecycle Summary

Creation -> Use -> Destruction

```
OBJECT LIFECYCLE:

1. CREATION
   - new operator allocates memory
   - Fields initialized to defaults
   - Constructor runs
   - Object becomes reachable

2. USAGE
   - Methods called, fields modified
   - References may be copied
```

```
   - Object remains in memory

3. ELIGIBLE FOR GC (when no references)
   - Reference set to null
   - Reference goes out of scope
   - Reference reassigned

4. COLLECTION (automatic or manual)
   - GC reclaims memory (Java/Python)
   - Destructor runs (C++)
   - Memory returned to system
```

Java Object Lifecycle

```java
class Person {
    String name;  // null default

    Person(String name) {  // Constructor
        this.name = name;
    }

    @Override
    protected void finalize() throws Throwable {
        // Called before GC (deprecated, don't use!)
        System.out.println(name + " is being finalized");
    }
}

// Creation
Person p = new Person("Alice");  // Object created on heap

// Usage
System.out.println(p.name);  // Using the object

// Ready for GC
p = null;  // No more references, eligible for collection

// System.gc() hints (doesn't force)
// JVM decides when to actually collect
```

Python Object Lifecycle

```python
import gc

class Person:
    def __init__(self, name):
        self.name = name
```

```
        print(f"{name} created")

    def __del__(self):
        # Destructor - called when object is about to be destroyed
        print(f"{self.name} destroyed")

# Creation
p = Person("Alice")  # "Alice created"

# Usage
print(p.name)

# Deletion
p = None  # or del p
# __del__ called before GC

# Force collection (usually not needed)
gc.collect()
```

C++ Object Lifecycle

```
class Person {
    string name;
public:
    Person(const string& n) : name(n) {
        cout << name << " constructed\n";
    }
    ~Person() {
        cout << name << " destroyed\n";
    }
};

int main() {
    // Stack object - auto cleanup
    Person p1("Alice");  // Constructed
} // "Alice destroyed" - automatic!

// Heap object - manual cleanup
Person* p2 = new Person("Bob");  // Constructed
delete p2;  // "Bob destroyed" - manual!

// Smart pointer - automatic
auto p3 = make_unique<Person>("Charlie");
// "Charlie destroyed" when p3 goes out of scope!
```

4.6 Memory Management in Different Languages

Java Memory Management

```java
// JVM Heap Structure:
// -Xms256m (initial heap)
// -Xmx1024m (max heap)
// -XX:NewRatio=2 (old/new ratio)

public class MemoryDemo {
    // Static (class area) - lives for entire program
    static int staticCounter;

    public void method() {
        // Stack frame created for method call
        int local = 10;  // On stack

        // Object on heap
        Object obj = new Object();
        // obj reference on stack, object on heap

        // When method returns:
        // - local is popped from stack
        // - obj eligible for GC (unless returned)
    }
}
```

Python Memory Management

```python
import sys

# Object allocation
x = object()

# Reference counting (immediate cleanup)
print(sys.getrefcount(x))  # Number of references

# Cycle detection for reference cycles
# gc module handles cycles that reference counting can't

# Memory view
import tracemalloc
tracemalloc.start()

# Your code here
tracemalloc.stop()
```

JavaScript Memory Management

```javascript
// V8 engine (Chrome, Node.js)
// Automatic garbage collection

let obj = { name: "test" };  // Object allocated
obj = null;  // Eligible for GC

// Weak references (ES2021)
let weakRef = new WeakRef(obj);
WeakRef.deref() // Get object or undefined

// Memory leak examples in JS:
// 1. Forgotten intervals
// 2. Detached DOM nodes
// 3. Closures holding references
```

Rust Ownership Model

```rust
// Rust: Ownership + Borrowing instead of GC

fn main() {
    let s1 = String::from("hello");  // s1 owns the string
    let s2 = s1;  // s1 MOVES to s2, s1 no longer valid

    // s1.println(); // ERROR! s1 moved

    let s3 = String::from("world");
    let s4 = &s3;  // Borrow (reference)
    println!("{}", s4);  // s4 borrows s3

    // Borrow checker ensures no dangling references
    // Memory freed when owner goes out of scope (no GC!)
}

// Multiple ownership with Rc<T>
use std::rc::Rc;

let rc = Rc::new(String::from("shared"));
let rc2 = rc.clone();  // Reference count increases
// Dropped when last Rc is dropped
```

Comparison Table

Feature	Java	Python	C++	Rust
GC Type	Multiple algorithms	Ref counting + cycle	Manual/RAII	Ownership
Stack/Heap	Both	Both	Both	Both
Memory Leaks	Possible (retained refs)	Possible (refs)	Possible (no delete)	Prevented by borrow checker
Deterministic cleanup	No	No	Yes (destructor)	Yes (RAII)
Escape analysis	Yes	Yes	Yes	Yes

4.7 Performance and Tuning

GC Tuning (Java Example)

```
# Common JVM flags:
java -Xms512m -Xmx2g  # Heap size
-XX:+UseG1GC          # Use G1 collector
-XX:MaxGCPauseMillis=200  # Target max pause
-XX:+PrintGCDetails   # GC logging

# Output analysis:
# java -XX:+PrintGCApplicationStoppedTime
# java -XX:+PrintGCDetails -Xloggc:gc.log
```

Memory Profiling Tools

```
// VisualVM (included in JDK)
// JConsole
// YourKit
// Java Flight Recorder

// Enable in code:
System.gc();  // Suggests GC (not guaranteed)
Runtime.getRuntime().freeMemory();  // Free memory
Runtime.getRuntime().totalMemory(); // Total memory
```

Python Memory Profiling

```
import tracemalloc

tracemalloc.start()

# Your code
snapshot = tracemalloc.take_snapshot()
```

```
top_stats = snapshot.statistics('lineno')

for stat in top_stats[:10]:
    print(stat)

# Memory leak detection
import objgraph
objgraph.show_most_common_types(limit=10)
objgraph.find_refchain(objgraph.by_type('MyClass')[0])
```

Best Practices

```
// DO:
- Create objects efficiently (avoid excessive allocation)
- Use primitive types when possible (no boxing)
- Clear references when done (set to null)
- Use try-with-resources for closeable resources
- Profile before optimizing GC settings

// DON'T:
- Create objects in loops unnecessarily
- Keep unnecessary references (leaks)
- Use System.gc() in production (use carefully)
- Prematurely optimize
```

Cheat-Sheet

Concept	Key Points
Stack	Fast, LIFO, local vars, auto cleanup
Heap	Slow, dynamic, objects, needs management
GC	Automatic memory reclamation
Reference counting	Immediate (Python), can't handle cycles
Mark and sweep	Marks reachable, sweeps garbage
Generational GC	Young/old generations, efficient
RAII	Resource tied to object lifetime (C++)
Smart pointers	auto-delete (unique/shared/weak)

Review Checklist

- ☐ **Stack vs Heap**: Can you explain when objects are allocated where?
- ☐ **GC process**: Do you understand mark-sweep-generational?
- ☐ **Eligibility**: Can you identify when objects are garbage?
- ☐ **Memory leaks**: Do you know common leak patterns?
- ☐ **Manual management**: Can you contrast C++ RAII vs Java GC?

☐ **Smart pointers**: Do you know unique vs shared vs weak?
☐ **Language differences**: Can you compare memory management approaches?
☐ **Performance**: Do you know basic GC tuning concepts?